🌿 KodeQuiz Book

Buku 6 — Coding Opsional: HTML, CSS, Java & PHP

23 bab · 250 soal dengan pembahasan

Cara memakai buku ini

1. Baca materi tiap bab (±3–5 menit).
2. Kerjakan *Uji Diri* tanpa melihat kunci — di kepala atau di kertas.
3. Cocokkan dengan *Kunci & Pembahasan* di akhir bab. Baca juga kenapa pilihan lain salah.
4. Soal yang salah? Tandai, lalu ulangi besok di aplikasi lewat Ulangan Pintar.

Dibuat 1 November 2026

Daftar Isi

1. HTML

1. Struktur Dasar Dokumen HTML
2. Teks, Heading & Daftar
3. Link, Gambar & Media
4. Form & Input
5. Semantik & Lanjutan
6. Aksesibilitas & SEO Dasar

2. CSS

1. Dasar: Selector, Warna & Cara Kerja CSS
2. Tipografi: Font & Teks
3. Box Model: Padding, Margin, Border
4. Layout Modern: Display & Flexbox
5. Position & CSS Grid
6. Efek, Animasi & Responsif
7. Spesifisitas, Variabel & Responsif Modern

3. Java

1. Dasar Java: Struktur Program & Tipe Data
2. Kontrol Alur: if, switch, for, while
3. Array & String
4. OOP: Class, Object, Inheritance, Interface
5. Koleksi (ArrayList, HashMap), Exception & Stream

4. PHP

1. Dasar PHP: Variabel, Teks & Cara Kerjanya
2. Kontrol Alur & Array
3. Fungsi & Fungsi Array Bawaan
4. PHP untuk Web: Form, Session & Keamanan
5. OOP, Database (PDO) & Laravel

BAGIAN 1

HTML

Bukan prioritas (opsional) — Cukup sampai bisa menyelesaikan proyek SalesKu dan halaman portofolio.

Struktur, form, semantik, aksesibilitas & SEO

6 bab · 61 soal

Struktur Dasar Dokumen HTML

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Setiap halaman web punya kerangka yang sama:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Judul di tab browser</title>
  </head>
  <body>
    <!-- semua konten yang terlihat di sini -->
  </body>
</html>
```

`<head>` berisi metadata (tak terlihat); `<body>` berisi konten. Komentar ditulis `<!-- … -->`.

🧠 Uji Diri — 7 soal

1. **HTML adalah singkatan dari…**

   A. HyperText Markup Language
   B. HighText Machine Language
   C. HyperTool Multi Language
   D. Home Text Markup Language

2. **Deklarasi yang benar di awal dokumen HTML5:**

   A. <!DOCTYPE html>
   B. <doctype html5>
   C. <html5>
   D. <!HTML>

3. **Konten yang tampil di tab browser ditulis di dalam tag...**

   A. <meta>
   B. <title>
   C. <header>
   D. <h1>

4. **Bagian dokumen yang berisi konten yang terlihat pengguna:**

   A. <head>
   B. <body>
   C. <main> saja
   D. <html>

5. **Cara menulis komentar di HTML:**

   A. // komentar
   B. # komentar
   C. <!-- komentar -->
   D. /* komentar */

6. **Tag pembuka untuk seluruh dokumen web adalah <____>**

   Jawab: _____

7. **Bagian tak terlihat berisi <title> dan <meta> adalah tag <____>**

   Jawab: _____

Kunci & Pembahasan · HTML Bab 1

← Kembali ke soal · Awal bab

1. **A. HyperText Markup Language** — HTML = HyperText Markup Language, bahasa markah halaman web.
2. **A. <!DOCTYPE html>** — <!DOCTYPE html> memberitahu browser bahwa dokumen adalah HTML5.
3. **B. <title>** — <title> di dalam <head> menentukan judul tab.
4. **B. <body>** — Semua konten tampak berada dalam <body>.
5. **C. <!-- komentar -->** — Komentar HTML diapit <!-- dan -->.
6. **html** — Semua konten dibungkus <html>...</html>.
7. **head** — <head> berisi metadata dokumen.

Teks, Heading & Daftar

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Heading bertingkat `<h1>` (terbesar) sampai `<h6>`; paragraf memakai `<p>`; pindah baris `<br>`; garis pemisah `<hr>`.

```
<strong>penting (tebal)</strong>
<em>penekanan (miring)</em>

<ul><li>bullet</li></ul>
<ol><li>bernomor</li></ol>
```

Gunakan `<strong>/<em>` (bermakna) ketimbang `<b>/<i>` (hanya visual). Tabel: `<table>` berisi baris `<tr>`, sel `<td>`, dan header sel `<th>`.

🧠 Uji Diri — 11 soal

1. **Tag untuk judul terbesar adalah…**

   A. <h6>
   B. <head>
   C. <h1>
   D. <heading>

2. **Tag untuk membuat paragraf:**

   A. <par>
   B. <p>
   C. <text>
   D. <pg>

3. **Tag untuk pindah baris (line break):**

   A. <lb>
   B. <break>
   C.
   D. <newline>

4. **Tag untuk daftar bernomor (1, 2, 3):**

   A.
   B.
   C.
   D. <dl>

5. **Setiap item dalam daftar ditulis dengan tag...**

   A. <item>
   B.
   C. <list>
   D. <it>

6. **Tag untuk teks tebal secara semantik (penting):**

   A. <b> selalu
   B.
   C. <bold>
   D.

7. **Tag untuk membuat tabel:**

   A. <tab>
   B. <table>
   C. <grid>
   D. <tbl>

8. **<tr> pada tabel berarti...**

   A. table row (baris)
   B. table record
   C. table right
   D. top row

9. **Tag daftar tak bernomor (bullet) adalah <____>**

   Jawab: _____

10. **Tag untuk teks miring secara semantik (penekanan) adalah <____>**

    Jawab: _____

11. **Tag untuk garis horizontal pemisah adalah <____>**

    Jawab: _____

Kunci & Pembahasan · HTML Bab 2

← Kembali ke soal · Awal bab

1. **C. <h1>** — <h1> adalah heading tingkat tertinggi; <h6> yang terkecil.
2. **B. <p>** — <p> mendefinisikan paragraf.
3. **C.
** —
 membuat baris baru dan tidak punya tag penutup.
4. **C. ** — = ordered list; = unordered (bullet).
5. **B. ** — = list item, dipakai di dalam <ul> maupun <ol>.
6. **B. ** — menandai teks penting; <b> hanya visual tanpa makna.
7. **B. <table>** — <table> dengan <tr> (baris), <td> (sel), <th> (header sel).
8. **A. table row (baris)** — <tr> = table row, satu baris tabel.
9. **ul** — = unordered list.
10. **em** — memberi penekanan; <i> hanya visual.
11. **hr** — <hr> membuat garis horizontal (thematic break).

Link, Gambar & Media

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
<a href="halaman2.html" target="_blank">Buka</a>
<img src="foto.jpg" alt="deskripsi gambar">
<video src="v.mp4" controls></video>
<iframe src="https://..."></iframe>
```

`href` = tujuan link (inilah cara menghubungkan antar page!); `target="_blank"` membuka tab baru. `alt` pada gambar wajib untuk aksesibilitas & tampil saat gambar gagal dimuat.

🧠 Uji Diri — 9 soal

1. **Tag untuk membuat tautan (link):**

   A. <link>
   B. <a>
   C. <href>
   D. <url>

2. **Atribut untuk alamat tujuan pada tag <a>:**

   A. src
   B. link
   C. href
   D. url

3. **Tag untuk menampilkan gambar:**

   A. <image>
   B. <picture> saja
   C. <img>
   D. <pic>

4. **Atribut wajib yang menunjukkan sumber gambar pada <img>:**

   A. href
   B. src
   C. alt
   D. source

5. **Atribut alt pada <img> berguna untuk...**

   A. mengganti ukuran
   B. aksesibilitas & teks jika gambar gagal dimuat
   C. animasi
   D. memberi border

6. **Atribut untuk membuka link di tab baru:**

   A. target="_blank"
   B. newtab="true"
   C. target="new"
   D. open="tab"

7. **Tag untuk memutar video di HTML5:**

   A. <media>
   B. <video>
   C. <movie>
   D. <play>

8. **Elemen <iframe> berfungsi untuk...**

   A. frame animasi
   B. menyematkan halaman/dokumen lain di dalam halaman
   C. membuat border
   D. format internal

9. **Atribut pada <img> untuk teks alternatif adalah ____**

   Jawab: _____

Kunci & Pembahasan · HTML Bab 3

← Kembali ke soal · Awal bab

1. **B. <a>** — <a href="..."> membuat hyperlink; <link> untuk memuat resource seperti CSS.
2. **C. href** — href (hypertext reference) berisi URL tujuan.
3. **C. <img>** — <img src="..." alt="..."> menampilkan gambar.
4. **B. src** — src menentukan file gambar; alt untuk teks alternatif.
5. **B. aksesibilitas & teks jika gambar gagal dimuat** — alt dibaca screen reader dan tampil saat gambar gagal dimuat.
6. **A. target="_blank"** — target="_blank" membuka tautan di tab/jendela baru.
7. **B. <video>** — <video controls src="..."> memutar video tanpa plugin.
8. **B. menyematkan halaman/dokumen lain di dalam halaman** — <iframe> menampilkan dokumen HTML lain, mis. video YouTube atau peta.
9. **alt** — alt penting untuk aksesibilitas dan SEO.

Form & Input

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Form mengumpulkan data pengguna:

```
<form action="/kirim" method="post">
  <label for="email">Email</label>
  <input type="email" id="email"
placeholder="kamu@mail.com" required>
  <input type="password">
  <select><option>Pilihan</option></select>
  <button type="submit">Kirim</button>
</form>
```

`method="get"` menaruh data di URL; `post` di body (untuk data sensitif). `required` = wajib diisi; `label for` terhubung ke `id` input.

🧠 Uji Diri — 9 soal

1. **Tag input yang benar untuk kolom kata sandi:**

   A. <input type="pass">
   B. <input type="password">
   C. <password>
   D. <input type="secret">

2. **Tag untuk mengelompokkan input dalam formulir:**

   A. <form>
   B. <fieldform>
   C. <inputgroup>
   D. <group>

3. **Elemen untuk pilihan dropdown:**

   A. <dropdown>
   B. <select> dengan <option>
   C. <list>
   D. <input type="drop">

4. **Atribut placeholder pada input berfungsi...**

   A. nilai default terkirim
   B. teks petunjuk samar sebelum diisi
   C. validasi
   D. label wajib

5. **Perbedaan method GET dan POST pada form:**

   A. Sama saja
   B. GET menaruh data di URL, POST di body request
   C. POST menaruh data di URL
   D. GET lebih aman untuk password

6. **Atribut required pada input berarti...**

   A. input dibaca saja
   B. wajib diisi sebelum form dikirim
   C. otomatis fokus
   D. terenkripsi

7. **Elemen <label> sebaiknya dihubungkan ke input lewat atribut...**

   A. name
   B. for (mengacu ke id input)
   C. link
   D. bind

8. **Input type email berbeda dari text karena...**

   A. tampilan beda warna
   B. browser memvalidasi format email otomatis
   C. hanya huruf kecil
   D. otomatis terkirim

9. **Tombol kirim form: <button type="____">**

   Jawab: _____

Kunci & Pembahasan · HTML Bab 4

← Kembali ke soal · Awal bab

1. **B. <input type="password">** — type="password" menyembunyikan karakter yang diketik.
2. **A. <form>** — <form> membungkus elemen input dan menentukan action & method.
3. **B. <select> dengan <option>** — <select> berisi beberapa <option>.
4. **B. teks petunjuk samar sebelum diisi** — placeholder menampilkan contoh/petunjuk dan hilang saat mengetik.
5. **B. GET menaruh data di URL, POST di body request** — GET terlihat di URL (cocok pencarian); POST untuk data sensitif/besar.
6. **B. wajib diisi sebelum form dikirim** — required memaksa pengguna mengisi field sebelum submit.
7. **B. for (mengacu ke id input)** — <label for="email"> terhubung ke <input id="email"> — penting untuk aksesibilitas.
8. **B. browser memvalidasi format email otomatis** — type="email" memberi validasi bawaan & keyboard khusus di mobile.
9. **submit** — type="submit" mengirim form saat diklik.

Semantik & Lanjutan

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Tag semantik memberi MAKNA pada struktur — bagus untuk SEO & aksesibilitas: `<header> <nav> <main> <article> <footer>`. Hanya boleh satu `<main>` per halaman.

Konsep penting: elemen **block** (div, p) vs **inline** (span, a); `id` harus unik, `class` boleh berulang; `data-*` menyimpan data kustom untuk JS.

```
<meta name="viewport" content="width=device-width,
initial-scale=1">
<link rel="stylesheet" href="style.css">
<script src="app.js" defer></script>
```

Script terbaik diberi `defer` atau ditaruh sebelum `</body>` agar halaman tampil dulu.

🧠 Uji Diri — 14 soal

1. **Tag semantik untuk navigasi utama situs:**

   A. <navigate>
   B. <menu>
   C. <nav>
   D. <links>

2. **Perbedaan elemen block dan inline:**

   A. Tidak ada bedanya
   B. Block memulai baris baru & selebar kontainer; inline mengikuti alur teks
   C. Inline selalu lebih besar
   D. Block hanya untuk teks

3. **Tag generik inline untuk membungkus sebagian teks:**

   A.
   B.
   C. <wrap>
   D. <sec>

4. **Atribut id berbeda dengan class karena...**

   A. id boleh dipakai berulang
   B. id harus unik dalam satu halaman
   C. class harus unik
   D. tidak ada bedanya

5. **Meta tag agar tampilan responsif di perangkat mobile:**

   A. <meta charset>
   B. <meta name="viewport" content="width=device-width, initial-scale=1">
   C. <meta responsive>
   D. <meta mobile="true">

6. **Tag semantik yang paling tepat untuk konten utama halaman:**

   A. <content>
   B. <main>
   C. <section> selalu
   D. <body>

7. **Tag semantik untuk konten mandiri seperti postingan blog:**

   A. <article>
   B. <aside>
   C.
   D. <post>

8. **Tag untuk menampilkan potongan kode program:**

   A. <code>
   B. <script>
   C. <program>
   D. <kbd> selalu

9. **Storage HTML5 untuk menyimpan data di browser tanpa kedaluwarsa:**

   A. cookies
   B. sessionStorage
   C. localStorage
   D. cache

10. **Atribut data-* digunakan untuk...**

    A. database
    B. menyimpan data kustom pada elemen
    C. enkripsi
    D. styling

11. **Posisi terbaik memuat <script> agar halaman tampil dulu:**

    A. di awal <head> tanpa atribut
    B. sebelum </body> atau memakai defer
    C. di dalam <title>
    D. di <meta>

12. **Tag semantik untuk bagian kaki halaman adalah <____>**

    Jawab: _____

13. **Tag untuk memuat file JavaScript eksternal: <____ src="app.js">**

    Jawab: _____

14. **Tag untuk menghubungkan file CSS eksternal: <____ rel="stylesheet" href="style.css">**

    Jawab: _____

Kunci & Pembahasan · HTML Bab 5

← Kembali ke soal · Awal bab

1. **C. <nav>** — <nav> menandai blok tautan navigasi.
2. **B. Block memulai baris baru & selebar kontainer; inline mengikuti alur teks** — Contoh block: div, p, h1. Contoh inline: span, a, img.
3. **B. ** — inline; block.
4. **B. id harus unik dalam satu halaman** — id unik per halaman; class boleh dipakai banyak elemen.
5. **B. <meta name="viewport" content="width=device-width, initial-scale=1">** — Viewport meta membuat lebar halaman mengikuti lebar layar perangkat.
6. **B. <main>** — <main> menandai konten utama; hanya boleh satu per halaman.
7. **A. <article>** — <article> untuk konten yang berdiri sendiri dan bisa didistribusikan ulang.
8. **A. <code>** — <code> untuk kode; sering dibungkus <pre> agar format terjaga.
9. **C. localStorage** — localStorage bertahan meski browser ditutup; sessionStorage hilang saat tab ditutup.
10. **B. menyimpan data kustom pada elemen** — data-* menyimpan informasi tambahan yang bisa dibaca JavaScript lewat dataset.

11. **B. sebelum </body> atau memakai defer** — Script di akhir body atau dengan defer tidak memblokir render HTML.
12. **footer** — <footer> berisi copyright, kontak, tautan bawah.
13. **script** — <script src="..."></script> memuat JS eksternal.
14. **link** — <link rel="stylesheet"> memuat CSS eksternal di <head>.

Aksesibilitas & SEO Dasar

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Halaman yang **aksesibel** bisa dipakai semua orang — termasuk pengguna keyboard dan screen reader. Bonusnya: struktur yang sama juga disukai mesin pencari (SEO).

```
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1">
  <title>Rina — Segmentasi Pelanggan E-commerce</title>
  <meta name="description" content="Studi kasus
segmentasi pelanggan dengan K-Means & K-Medoids.">
</head>
<body>
  <h1>Judul utama (satu saja)</h1>
  <h2>Bagian</h2>  <h3>Sub-bagian</h3>

  <img src="grafik.png" alt="Grafik pendapatan naik 41%
Jan–Jun">

  <label for="email">Email</label>
  <input id="email" type="email">

  <button aria-label="Hapus transaksi">🗑</button>
  <a href="laporan.pdf">Unduh laporan Q1 (PDF)</a>
</body>
```

Checklist cepat: semua gambar punya alt · semua input punya label · tombol memakai <button> · heading berurutan · teks link deskriptif · bisa dipakai hanya dengan keyboard (Tab, Enter) · kontras warna cukup.

🧪 Uji halamanmu: tekan Tab berulang kali tanpa mouse. Apakah fokusnya terlihat dan semua tombol bisa dipakai? Lalu jalankan Lighthouse di Chrome DevTools (tab Lighthouse) untuk skor aksesibilitas & SEO.

🧠 Uji Diri — 11 soal

1. **Fungsi utama atribut alt pada <img>:**

```
<img src="grafik-penjualan.png" alt="Grafik penjualan Januari–Juni naik 41%">
```

   A. Teks pengganti yang dibacakan screen reader dan tampil jika gambar gagal dimuat; juga dibaca mesin pencari
   B. Judul yang muncul saat mouse diarahkan
   C. Ukuran gambar
   D. Nama file cadangan

2. **Cara yang benar menghubungkan label dengan input:**

   A. <label for="email">Email</label> <input id="email" type="email">
   B. Email <input type="email">
   C. <input placeholder="Email"> saja
   D. <label>Email</label> di halaman lain

3. **Kenapa <button> lebih baik daripada untuk tombol?**

   A. <button> otomatis bisa difokus dengan Tab, ditekan dengan Enter/Spasi, dan dikenali screen reader sebagai tombol
   B. tidak bisa diberi warna
   C. <button> lebih cepat dimuat
   D. Tidak ada bedanya

4. **Struktur heading yang baik untuk satu halaman:**

   A. Satu <h1> sebagai judul utama, lalu <h2> untuk bagian dan <h3> untuk sub-bagian — tanpa melompati level
   B. Semua judul memakai <h1> agar besar
   C. Memilih <h4> karena ukurannya pas
   D. Tidak memakai heading, cukup <b>

5. **Tag yang menentukan deskripsi halaman di hasil pencarian Google:**

   A. <meta name="description" content="...">
   B. <title>
   C. <h1>
   D. <footer>

6. **Atribut lang="id" pada <html lang="id"> berguna untuk...**

   A. Memberi tahu browser, screen reader, dan mesin pencari bahwa bahasa halaman adalah Bahasa Indonesia
   B. Menerjemahkan halaman otomatis
   C. Mengubah font
   D. Mengatur zona waktu

7. **Tombol yang hanya berisi ikon (misal 🗑) sebaiknya diberi...**

```
<button aria-label="Hapus transaksi">🗑</button>
```

   A. aria-label yang menjelaskan fungsinya
   B. Warna merah saja
   C. Ukuran yang lebih besar saja
   D. Tidak perlu apa-apa

8. **Teks link mana yang paling baik?**

   A. "Unduh laporan penjualan Q1 (PDF)"
   B. "Klik di sini"
   C. "Link"
   D. "https://contoh.com/files/laporan_q1_final_v2.pdf"

9. **Tabel data yang aksesibel sebaiknya memakai...**

   A. <caption> untuk judul tabel dan <th scope="col"> untuk judul kolom
   B. Hanya <td> dengan teks tebal
   C. Gambar tabel (screenshot)
   D. yang disusun seperti tabel

10. **Tanpa baris ini, halaman di HP tampil seperti versi desktop yang diperkecil:**

    A. <meta name="viewport" content="width=device-width, initial-scale=1">
    B. <meta charset="UTF-8">
    C. <link rel="icon">
    D. <meta name="author">

11. **Atribut <img> untuk teks alternatif gambar: ___**

    Jawab: _____

Kunci & Pembahasan · HTML Bab 6

← Kembali ke soal · Awal bab

1. **A. Teks pengganti yang dibacakan screen reader dan tampil jika gambar gagal dimuat; juga dibaca mesin pencari** — alt membuat gambar bisa 'dibaca' oleh tunanetra dan mesin pencari. Tulis isinya/maknanya, bukan 'gambar1.png'. Untuk gambar dekoratif, isi alt="" supaya dilewati.
   - **B:** Itu atribut title.
   - **C:** Ukuran diatur dengan width/height atau CSS.
   - **D:** Tidak ada fungsi file cadangan di alt.
2. **A. <label for="email">Email</label> <input id="email" type="email">** — for harus sama dengan id input. Hasilnya: klik teks label memfokuskan input (area klik lebih besar di HP), dan screen reader membacakan nama field-nya. Placeholder bukan pengganti label karena hilang saat mengetik.
   - **B:** Paragraf biasa tidak terhubung dengan input.
   - **C:** Placeholder hilang saat user mulai mengetik.
   - **D:** Label harus berada di form yang sama dan terhubung lewat for/id.

3. **A. <button> otomatis bisa difokus dengan Tab, ditekan dengan Enter/Spasi, dan dikenali screen reader sebagai tombol** — Elemen semantik memberi aksesibilitas gratis. Membuat berperilaku seperti tombol butuh tabindex, role, dan event keyboard tambahan — dan sering lupa.
    - **B:** div bisa diberi gaya apa pun.
    - **C:** Kecepatannya sama.
    - **D:** Bedanya besar untuk pengguna keyboard dan screen reader.

4. **A. Satu <h1> sebagai judul utama, lalu <h2> untuk bagian dan <h3> untuk sub-bagian — tanpa melompati level** — Heading adalah 'daftar isi' halaman bagi screen reader dan mesin pencari. Ukuran tampilan diatur dengan CSS, bukan dengan memilih level heading.
    - **B:** Banyak h1 mengaburkan judul utama.
    - **C:** Level heading menunjukkan struktur, bukan ukuran.
    - **D:** <b> tidak memberi struktur dokumen.

5. **A. <meta name="description" content="..."> —** Meta description (±150 karakter) sering tampil di bawah judul hasil pencarian dan saat link dibagikan. <title> menentukan judul tab dan judul hasil pencarian. Template portofolio di proyek ini sudah menyediakan keduanya.
    - **B:** <title> adalah judulnya, bukan deskripsinya.
    - **C:** <h1> adalah judul di dalam halaman.
    - **D:** Footer adalah bagian bawah halaman.

6. **A. Memberi tahu browser, screen reader, dan mesin pencari bahwa bahasa halaman adalah Bahasa Indonesia** — Screen reader memilih pelafalan yang benar, browser menawarkan terjemahan dengan tepat, dan mesin pencari memahami target bahasanya.
    - **B:** lang tidak menerjemahkan apa pun.
    - **C:** Font diatur dengan CSS.
    - **D:** Zona waktu tidak berhubungan dengan HTML.

7. **A. aria-label yang menjelaskan fungsinya** — Tanpa teks, screen reader hanya membacakan 'button' atau nama emoji. aria-label memberi nama yang bermakna. Atribut title juga membantu pengguna mouse (tooltip).
    - **B:** Warna tidak terbaca oleh screen reader.
    - **C:** Ukuran tidak memberi nama tombol.
    - **D:** Tombol tanpa nama membingungkan pengguna screen reader.

8. **A. "Unduh laporan penjualan Q1 (PDF)"** — Pengguna screen reader sering melompat dari link ke link. 'Klik di sini' tidak bermakna tanpa konteks. Teks link yang deskriptif juga membantu SEO.
    - **B:** Tidak menjelaskan tujuan link.
    - **C:** Tidak bermakna.
    - **D:** URL panjang sulit dibaca dan dibacakan.

9. **A. <caption> untuk judul tabel dan <th scope="col"> untuk judul kolom** — th dan scope memberi tahu screen reader judul kolom/baris setiap sel. Gambar tabel tidak bisa dibaca, dicari, maupun disalin.
    - **B:** Teks tebal tidak menjelaskan struktur.
    - **C:** Gambar tidak bisa dibaca screen reader dan mesin pencari.
    - **D:** div tidak punya makna tabel.
10. **A. <meta name="viewport" content="width=device-width, initial-scale=1">** — Meta viewport memberi tahu browser HP agar lebar halaman mengikuti lebar layar. Wajib untuk halaman responsif, termasuk portofolio dan proyek SalesKu.
    - **B:** charset mengatur penyandian karakter.
    - **C:** icon mengatur favicon.
    - **D:** author hanya informasi pembuat.
11. **alt** — alt="deskripsi singkat". Gambar dekoratif: alt="" agar dilewati screen reader.

BAGIAN 2

CSS

Bukan prioritas (opsional) — Secukupnya untuk merapikan proyek dan portofolio.

Box model, Flexbox, Grid, variabel, responsif

7 bab · 111 soal

Dasar: Selector, Warna & Cara Kerja CSS

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Aturan CSS = **selector** + properti:

```
.kartu { color: #333; }   /* class  → .nama  */
#menu  { color: red; }    /* id     → #nama  */
p, h1  { margin: 0; }     /* grup (koma)     */
div p  { }                /* p di dalam div  */
div > p { }               /* anak langsung   */
```

Warna: nama (`red`), hex (`#4c8dff`), `rgb()`, `rgba()` dengan transparansi. Jika dua aturan bentrok, yang menang: specificity lebih tinggi (id > class > tag), lalu yang ditulis terakhir. CSS terbaik diletakkan di file eksternal via `<link>`.

🧠 Uji Diri — 18 soal

1. **CSS singkatan dari...**

   A. Creative Style Sheets
   B. Cascading Style Sheets
   C. Computer Style Sheets
   D. Colorful Style Sheets

2. **Selector untuk class "kotak":**

   A. #kotak
   B. kotak
   C. .kotak
   D. *kotak

3. **Selector untuk id "menu":**

   A. .menu
   B. #menu
   C. menu
   D. @menu

4. **Properti warna teks:**

   A. text-color
   B. font-color
   C. color
   D. text-style

5. **Properti warna latar belakang:**

   A. bg-color
   B. background-color
   C. color-bg
   D. back-color

6. **Cara menulis komentar CSS:**

   A. // komentar
   B. <!-- -->
   C. /* komentar */
   D. # komentar

7. **Cara terbaik memakai CSS di proyek besar:**

   A. inline style
   B. <style> tiap halaman
   C. file eksternal via <link>
   D. atribut css

8. **#ffffff menghasilkan warna...**

   A. hitam
   B. putih
   C. merah
   D. abu

9. **Selector p, h1 { } berarti...**

   A. p di dalam h1
   B. p ATAU h1 (keduanya kena)
   C. p setelah h1
   D. error

10. **Selector div p { } (dengan spasi) memilih...**

    A. div dan p
    B. semua p di dalam div (berapa pun dalamnya)
    C. p anak langsung
    D. div setelah p

11. **Selector div > p memilih...**

    A. semua p dalam div
    B. p yang anak LANGSUNG dari div
    C. div dalam p
    D. p pertama

12. **Selector * { } berlaku untuk...**

    A. elemen pertama
    B. semua elemen
    C. body saja
    D. class *

13. **rgb(255, 0, 0) adalah warna...**

    A. biru
    B. hijau
    C. merah
    D. kuning

14. **Urutan prioritas (specificity) tertinggi ke rendah:**

    A. tag > class > id
    B. id > class > tag
    C. class > id > tag
    D. semua sama

15. **Jika dua aturan sama kuat, yang menang adalah...**

    A. yang pertama
    B. yang ditulis paling akhir
    C. yang terpendek
    D. acak

16. **Tag HTML untuk memuat CSS eksternal: <____ rel="stylesheet" href="style.css">**

    Jawab: _____

17. **Kata kunci pemaksa prioritas: color: red !____;**

    Jawab: _____

18. **rgba(0,0,0,0.5) — huruf "a" berarti ____ (transparansi)**

    Jawab: _____

Kunci & Pembahasan · CSS Bab 1

← Kembali ke soal · Awal bab

1. **B. Cascading Style Sheets** — Cascading Style Sheets — pengatur tampilan halaman.
2. **C. .kotak** — Class diawali titik (.), id diawali pagar (#).
3. **B. #menu** — # untuk id.
4. **C. color** — color mengatur warna teks.
5. **B. background-color** — background-color.
6. **C. /* komentar */** — CSS hanya mengenal /* ... */.
7. **C. file eksternal via <link>** — CSS eksternal bisa dipakai banyak halaman & mudah dirawat.
8. **B. putih** — #ffffff putih; #000000 hitam.
9. **B. p ATAU h1 (keduanya kena)** — Koma = grup selector, aturan berlaku untuk semuanya.
10. **B. semua p di dalam div (berapa pun dalamnya)** — Spasi = descendant selector.
11. **B. p yang anak LANGSUNG dari div** — > = child combinator (anak langsung).
12. **B. semua elemen** — * = universal selector.
13. **C. merah** — Urutan RGB: Red, Green, Blue.
14. **B. id > class > tag** — Inline > id > class > tag; !important menimpa semua.
15. **B. yang ditulis paling akhir** — Itulah makna 'cascading' — aturan terakhir menang.

16. **link** — <link> diletakkan di <head>.
17. **important** — !important menimpa aturan lain — pakai seperlunya.
18. **alpha** — Alpha: 0 transparan, 1 solid.

Tipografi: Font & Teks

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Properti teks yang paling sering dipakai:

```
p {
  font-family: 'Poppins', sans-serif; /* + fallback */
  font-size: 16px;
  font-weight: 700;        /* 400 normal, 700 bold */
  line-height: 1.6;        /* jarak antar baris    */
  text-align: center;
  text-transform: uppercase;
  text-decoration: none;   /* hapus underline link */
  letter-spacing: 1px;
}
```

Tips: `line-height 1.5–1.7` membuat paragraf nyaman dibaca — rahasia website yang enak dipandang.

🧠 Uji Diri — 15 soal

1. **Properti ukuran huruf:**

   A. text-size
   B. font-size
   C. size
   D. font-height

2. **Properti jenis huruf:**

   A. font-family
   B. font-type
   C. text-font
   D. typeface

3. **Membuat teks tebal:**

   A. font-weight: bold
   B. text-bold: true
   C. font-style: bold
   D. weight: heavy

4. **Membuat teks miring:**

   A. font-style: italic
   B. text-style: italic
   C. font-italic: true
   D. style: slant

5. **Teks rata tengah:**

   A. align: center
   B. text-align: center
   C. center: true
   D. margin: center

6. **Menghapus garis bawah link:**

   A. underline: none
   B. text-decoration: none
   C. link-style: plain
   D. border: none

7. **Semua huruf jadi KAPITAL via CSS:**

   A. font-case: upper
   B. text-transform: uppercase
   C. text-style: caps
   D. font-upper: on

8. **Properti jarak antar baris teks:**

   A. line-height
   B. line-space
   C. row-gap
   D. text-gap

9. **Properti jarak antar huruf:**

   A. char-gap
   B. letter-spacing
   C. font-spacing
   D. kerning-size

10. **font-family: 'Poppins', sans-serif; — sans-serif berperan sebagai...**

    A. dekorasi
    B. font cadangan (fallback)
    C. ukuran
    D. varian tebal

11. **font-weight: 400 setara dengan...**

    A. bold
    B. normal
    C. light
    D. black

12. **Memberi bayangan pada TEKS:**

    A. box-shadow
    B. text-shadow
    C. font-shadow
    D. shadow

13. **Memotong teks kepanjangan dengan "...":**

    A. text-cut: dots
    B. text-overflow: ellipsis (+ overflow:hidden, white-space:nowrap)
    C. font-trim: auto
    D. clip: text

14. **Garis bawah teks: text-decoration: ____;**

    Jawab: _____

15. **Huruf pertama tiap kata kapital: text-transform: ____;**

    Jawab: _____

Kunci & Pembahasan · CSS Bab 2

← Kembali ke soal · Awal bab

1. **B. font-size** — font-size: 16px;
2. **A. font-family** — font-family: 'Poppins', sans-serif;
3. **A. font-weight: bold** — font-weight: bold atau angka 700.
4. **A. font-style: italic** — font-style: italic.
5. **B. text-align: center** — text-align: left/center/right/justify.
6. **B. text-decoration: none** — text-decoration: none menghilangkan underline.
7. **B. text-transform: uppercase** — text-transform: uppercase/lowercase/capitalize.
8. **A. line-height** — line-height mengatur tinggi baris.
9. **B. letter-spacing** — letter-spacing: 1px;
10. **B. font cadangan (fallback)** — Fallback dipakai jika font utama gagal dimuat.
11. **B. normal** — 400 = normal; 700 = bold.
12. **B. text-shadow** — text-shadow: x y blur warna.
13. **B. text-overflow: ellipsis (+ overflow:hidden, white-space:nowrap)** — Trio properti itu menghasilkan elipsis (...).
14. **underline** — underline memberi garis bawah.
15. **capitalize** — capitalize mengkapitalkan awal kata.

Box Model: Padding, Margin, Border

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Setiap elemen adalah kotak berlapis: **konten → padding → border → margin**.

```
.kartu {
  padding: 16px;          /* ruang DALAM  */
  margin: 10px 20px;      /* atas-bawah 10, kiri-kanan
20 */
  border: 1px solid #ddd;
  border-radius: 12px;    /* sudut lengkung; 50% =
lingkaran */
  max-width: 600px;
  margin-inline: auto;    /* tengah horizontal */
}
```

Empat nilai margin/padding urut jarum jam: atas-kanan-bawah-kiri. Selalu pakai `box-sizing: border-box` agar width sudah termasuk padding & border — ukuran jadi mudah diprediksi.

🧠 Uji Diri — 17 soal

1. **Jarak DI DALAM elemen (konten ↔ border):**

   A. margin
   B. spacing
   C. padding
   D. gap

2. **Jarak DI LUAR elemen:**

   A. padding
   B. margin
   C. border
   D. outline

3. **Properti garis tepi elemen:**

   A. line
   B. border
   C. outline saja
   D. edge

4. **Properti lebar elemen:**

   A. width
   B. wide
   C. size-x
   D. w

5. **Membuat sudut melengkung:**

   A. corner
   B. border-radius
   C. round
   D. radius-style

6. **margin: 10px 20px 30px 40px — urutannya...**

   A. kiri-atas-kanan-bawah
   B. atas-kanan-bawah-kiri
   C. atas-bawah-kiri-kanan
   D. acak

7. **margin: 10px 20px; berarti...**

   A. atas-bawah 10, kiri-kanan 20
   B. kiri 10, kanan 20
   C. semua 10
   D. semua 20

8. **Membuat elemen block berada di tengah horizontal:**

   A. margin: 0 auto
   B. align: center
   C. center: block
   D. float: center

9. **Urutan box model dari dalam ke luar:**

   A. content-padding-border-margin
   B. content-margin-border-padding
   C. padding-content-border-margin
   D. margin-border-content-padding

10. **border: 2px dashed red; — dashed adalah...**

    A. ketebalan
    B. gaya garis (putus-putus)
    C. warna
    D. radius

11. **box-sizing: border-box membuat...**

    A. border tebal
    B. width sudah TERMASUK padding & border
    C. margin masuk hitungan
    D. kotak bulat

12. **Tanpa border-box, elemen width:200px + padding:20px lebar totalnya...**

    A. 200px
    B. 220px
    C. 240px
    D. 180px

13. **max-width: 600px berguna agar...**

    A. minimal 600px
    B. elemen tak melebihi 600px tapi bisa menyempit
    C. tepat 600px
    D. tersembunyi

14. **overflow: hidden pada kontainer akan...**

    A. menampilkan scrollbar
    B. memotong konten yang meluap
    C. memperbesar konten
    D. error

15. **border-radius: 50% pada kotak persegi menghasilkan...**

    A. oval acak
    B. lingkaran
    C. segitiga
    D. tak berubah

16. **Properti tinggi elemen: ____: 100px;**

    Jawab: _____

17. **Menghilangkan garis tepi: border: ____;**

    Jawab: _____

Kunci & Pembahasan · CSS Bab 3

← Kembali ke soal · Awal bab

1. **C. padding** — padding ruang dalam; margin ruang luar.
2. **B. margin** — margin memberi jarak antar elemen.
3. **B. border** — border: 1px solid black;
4. **A. width** — width: 200px;
5. **B. border-radius** — border-radius: 8px;
6. **B. atas-kanan-bawah-kiri** — Searah jarum jam mulai atas.
7. **A. atas-bawah 10, kiri-kanan 20** — Dua nilai: vertikal lalu horizontal.
8. **A. margin: 0 auto** — margin auto kiri-kanan menengahkan block ber-width.
9. **A. content-padding-border-margin** — Konten → padding → border → margin.
10. **B. gaya garis (putus-putus)** — Gaya: solid, dashed, dotted, double.
11. **B. width sudah TERMASUK padding & border** — border-box membuat ukuran mudah diprediksi.
12. **C. 240px** — 200 + 20 kiri + 20 kanan = 240px.
13. **B. elemen tak melebihi 600px tapi bisa menyempit** — max-width kunci desain responsif konten.
14. **B. memotong konten yang meluap** — hidden memotong; auto memberi scrollbar bila perlu.

15. **B. lingkaran** — 50% pada persegi = lingkaran sempurna (foto profil!).
16. **height** — height mengatur tinggi.
17. **none** — border: none menghapus border.

Layout Modern: Display & Flexbox

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Block (div, p) memulai baris baru; **inline** (span, a) mengikuti alur teks. `display:none` menghilangkan elemen total.

Flexbox — cara modern menata satu dimensi:

```
.wadah {
  display: flex;
  justify-content: center;   /* sumbu utama   */
  align-items: center;       /* sumbu silang  */
  gap: 16px;                 /* jarak item    */
  flex-wrap: wrap;           /* turun baris   */
  flex-direction: column;    /* jadi vertikal */
}
```

Kombinasi `justify-content:center` + `align-items:center` = elemen tepat di tengah. `flex:1` pada item = mengisi sisa ruang.

🧠 Uji Diri — 18 soal

1. **Elemen yang selalu memulai baris baru disebut…**

   A. inline
   B. block
   C. flex
   D. float

2. **Menyembunyikan elemen sekaligus menghapus ruangnya:**

   A. visibility: hidden
   B. display: none
   C. opacity: 0
   D. hide: true

3. **Mengaktifkan flexbox pada kontainer:**

   A. display: flex
   B. flex: on
   C. layout: flex
   D. position: flex

4. **Flexbox mengatur item dalam...**

   A. dua dimensi
   B. satu dimensi (baris ATAU kolom)
   C. lingkaran
   D. tabel

5. **Properti flex untuk merapikan item di sumbu utama (horizontal default):**

   A. align-items
   B. justify-content
   C. text-align
   D. flex-align

6. **Properti flex untuk sumbu silang (vertikal default):**

   A. justify-content
   B. align-items
   C. valign
   D. cross-align

7. **Menengahkan elemen persis di tengah dengan flex:**

   A. margin center
   B. justify-content:center + align-items:center
   C. text-align saja
   D. position:middle

8. **flex-direction: column membuat item tersusun...**

   A. menyamping
   B. dari atas ke bawah
   C. terbalik
   D. melingkar

9. **Properti gap pada flex/grid:**

   A. padding item
   B. celah antar item
   C. margin kontainer
   D. lebar item

10. **justify-content: space-between menghasilkan...**

    A. item rapat kiri
    B. item menempel ujung-ujung, sisa ruang di antara
    C. semua di tengah
    D. item bertumpuk

11. **flex-wrap: wrap berguna agar...**

    A. item mengecil terus
    B. item turun ke baris baru jika tak muat
    C. item tersembunyi
    D. animasi

12. **flex: 1 pada item berarti...**

    A. urutan pertama
    B. mengisi sisa ruang yang tersedia
    C. lebar 1px
    D. tersembunyi

13. **display: inline-block berguna karena...**

    A. selalu baris baru
    B. sejajar seperti inline TAPI bisa diberi width/height
    C. tanpa margin
    D. lebih cepat

14. **Properti order pada flex item mengatur...**

    A. abjad
    B. urutan tampil tanpa mengubah HTML
    C. ukuran
    D. warna

15. **align-self berbeda dari align-items karena...**

    A. sama
    B. berlaku untuk SATU item saja
    C. untuk teks
    D. untuk grid saja

16. **display: ____ menghilangkan elemen sepenuhnya**

    Jawab: _____

17. **Menyusun flex item jadi vertikal: flex-direction: ____;**

    Jawab: _____

18. **Item flex turun baris jika sempit: flex-wrap: ____;**

    Jawab: _____

Kunci & Pembahasan · CSS Bab 4

← Kembali ke soal · Awal bab

1. **B. block** — Block: div, p, h1. Inline: span, a.
2. **B. display: none** — display:none hilang total; visibility:hidden menyisakan ruang.
3. **A. display: flex** — display: flex pada elemen induk.
4. **B. satu dimensi (baris ATAU kolom)** — Flex = 1 dimensi; Grid = 2 dimensi.
5. **B. justify-content** — justify-content: center/space-between/dst.
6. **B. align-items** — align-items: center menengahkan vertikal.
7. **B. justify-content:center + align-items:center** — Kombinasi keduanya = tengah sempurna.
8. **B. dari atas ke bawah** — column mengubah sumbu utama jadi vertikal.
9. **B. celah antar item** — gap memberi jarak antar item tanpa margin manual.
10. **B. item menempel ujung-ujung, sisa ruang di antara** — space-between menyebar item ke tepi.
11. **B. item turun ke baris baru jika tak muat** — wrap membuat layout fleksibel di layar sempit.
12. **B. mengisi sisa ruang yang tersedia** — flex:1 = tumbuh mengisi ruang kosong.
13. **B. sejajar seperti inline TAPI bisa diberi width/height** — Gabungan sifat inline dan block.

14. **B. urutan tampil tanpa mengubah HTML** — order memindah posisi visual item.
15. **B. berlaku untuk SATU item saja** — align-self menimpa align-items pada item tertentu.
16. **none** — display:none menghapus dari layout.
17. **column** — column = atas ke bawah.
18. **wrap** — wrap mengizinkan multi-baris.

Position & CSS Grid

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Position: `static` (default) → `relative` (geser dari posisi asal, jadi acuan anak) → `absolute` (lepas dari alur, relatif ke leluhur ber-position) → `fixed` (menempel layar) → `sticky` (menempel saat discroll). `z-index` mengatur tumpukan depan-belakang.

Grid untuk layout 2 dimensi:

```
.galeri {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 kolom
rata */
  gap: 16px;
}
.sidebar-layout { grid-template-columns: 200px 1fr; }
```

🧠 Uji Diri — 15 soal

1. **Nilai position default semua elemen:**

   A. relative
   B. absolute
   C. static
   D. fixed

2. **position: fixed membuat elemen…**

   A. ikut scroll
   B. menempel viewport, tak ikut scroll
   C. di tengah
   D. hilang

3. **position: absolute diposisikan relatif terhadap...**

   A. viewport selalu
   B. leluhur terdekat yang ber-position (biasanya relative)
   C. body selalu
   D. dirinya

4. **z-index mengatur...**

   A. zoom
   B. urutan tumpukan (depan/belakang)
   C. indeks list
   D. jarak

5. **position: sticky membuat elemen...**

   A. selalu diam
   B. normal, lalu menempel saat tergeser scroll
   C. absolute
   D. transparan

6. **Mengaktifkan CSS Grid:**

   A. display: grid
   B. grid: on
   C. layout: grid
   D. position: grid

7. **Grid unggul dibanding flex untuk...**

   A. satu baris
   B. layout dua dimensi (baris DAN kolom)
   C. teks
   D. animasi

8. **grid-template-columns: repeat(3, 1fr) menghasilkan...**

   A. 3 baris
   B. 3 kolom sama lebar
   C. 1 kolom
   D. 3px

9. **Satuan fr pada grid berarti...**

   A. frame
   B. fraksi dari ruang tersedia
   C. font-relative
   D. freeze

10. **grid-column: span 2 membuat item...**

    A. pindah kolom 2
    B. membentang selebar 2 kolom
    C. hilang
    D. 2px

11. **top/left/right/bottom hanya berpengaruh jika...**

    A. selalu
    B. position bukan static
    C. display flex
    D. ada margin

12. **grid-template-columns: 200px 1fr berarti...**

    A. dua kolom sama
    B. kolom kiri tetap 200px, kanan mengisi sisa
    C. satu kolom
    D. error

13. **position: ____ menjadikan elemen acuan untuk anak absolute**

    Jawab: _____

14. **Properti jarak antar sel grid: ____: 16px;**

    Jawab: _____

15. **display: ____ mengaktifkan layout dua dimensi**

    Jawab: _____

Kunci & Pembahasan · CSS Bab 5

← Kembali ke soal · Awal bab

1. **C. static** — static = mengikuti alur normal dokumen.
2. **B. menempel viewport, tak ikut scroll** — Cocok untuk navbar melayang.
3. **B. leluhur terdekat yang ber-position (biasanya relative)** — Karena itu induknya sering diberi position:relative.
4. **B. urutan tumpukan (depan/belakang)** — z-index besar = di depan (butuh position selain static).
5. **B. normal, lalu menempel saat tergeser scroll** — sticky = gabungan relative + fixed.
6. **A. display: grid** — display: grid pada kontainer.
7. **B. layout dua dimensi (baris DAN kolom)** — Grid mengontrol baris & kolom sekaligus.
8. **B. 3 kolom sama lebar** — 1fr = 1 bagian ruang; repeat(3,1fr) = 3 kolom rata.
9. **B. fraksi dari ruang tersedia** — fr membagi ruang kosong secara proporsional.
10. **B. membentang selebar 2 kolom** — span melebarkan item melintasi beberapa track.
11. **B. position bukan static** — Properti offset butuh position relative/absolute/fixed/sticky.
12. **B. kolom kiri tetap 200px, kanan mengisi sisa** — Pola sidebar klasik: tetap + fleksibel.

13. **relative** — relative membuat elemen jadi 'containing block'.
14. **gap** — gap berlaku di grid dan flex.
15. **grid** — CSS Grid untuk baris + kolom.

Efek, Animasi & Responsif

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Sentuhan yang membuat website terasa hidup:

```
.btn {
  box-shadow: 0 4px 12px rgba(0,0,0,.15);
  transition: all .3s ease;
  cursor: pointer;
}
.btn:hover { transform: scale(1.05); }
```

`:hover`, `:focus`, `::before/::after` adalah pseudo yang paling sering dipakai. Variabel CSS: `:root { --utama: #4c8dff; }` → `var(--utama)`.

Responsif — satuan relatif (`rem`, `%`, `vh/vw`) + media query:

```
@media (max-width: 600px) {
  .galeri { grid-template-columns: 1fr; }
}
```

🧠 Uji Diri — 17 soal

1. **Properti transparansi elemen:**

   A. transparent
   B. opacity
   C. alpha
   D. fade

2. **Bayangan pada kotak/elemen:**

   A. shadow
   B. box-shadow
   C. element-shadow
   D. blur

3. **Pseudo-class saat kursor di atas elemen:**

   A. :active
   B. :hover
   C. :focus
   D. :visited

4. **Pseudo-class input yang sedang aktif diketik:**

   A. :hover
   B. :focus
   C. :active
   D. :typing

5. **transition: all 0.3s ease; membuat...**

   A. animasi loop
   B. perubahan properti jadi halus 0.3 detik
   C. delay klik
   D. rotasi

6. **Gambar sebagai latar elemen:**

   A. img-bg
   B. background-image: url('x.jpg')
   C. src-bg
   D. image: back

7. **background-size: cover berarti...**

   A. gambar asli
   B. gambar menutupi penuh area (bisa terpotong)
   C. diulang-ulang
   D. diperkecil

8. **transform: translateX(50px) akan...**

   A. memutar
   B. menggeser 50px ke kanan
   C. memperbesar
   D. memiringkan

9. **transform: scale(1.1) saat hover menghasilkan efek...**

   A. berputar
   B. membesar 10% (zoom halus)
   C. memudar
   D. bergeser

10. **@keyframes digunakan untuk...**

    A. media query
    B. mendefinisikan tahapan animasi
    C. font
    D. variabel

11. **Cara membuat variabel CSS:**

    A. $warna: red
    B. --warna: red; dipakai var(--warna)
    C. @warna = red
    D. var warna

12. **Media query untuk layar maksimal 600px:**

    A. @media (max-width: 600px){ }
    B. @screen 600
    C. @media w=600
    D. @responsive

13. **Satuan relatif terhadap font elemen root:**

    A. px
    B. em
    C. rem
    D. %

14. **100vh berarti...**

    A. 100 huruf
    B. setinggi viewport penuh
    C. 100px
    D. 100% lebar

15. **Pseudo-element menambah konten dekoratif sebelum elemen:**

    A. ::before
    B. ::first
    C. ::pre
    D. :start

16. **cursor: ____ membuat kursor jadi tangan penunjuk**

    Jawab: _____

17. **Memutar elemen 45°: transform: ____(45deg);**

    Jawab: _____

Kunci & Pembahasan · CSS Bab 6

← Kembali ke soal · Awal bab

1. **B. opacity** — opacity: 0 (transparan) – 1 (solid).
2. **B. box-shadow** — box-shadow: x y blur warna.
3. **B. :hover** — :hover untuk efek sentuh kursor.
4. **B. :focus** — :focus saat elemen menerima fokus.
5. **B. perubahan properti jadi halus 0.3 detik** — transition menghaluskan perubahan (mis. saat hover).
6. **B. background-image: url('x.jpg')** — background-image + background-size: cover biasanya.
7. **B. gambar menutupi penuh area (bisa terpotong)** — cover mengisi penuh; contain memuat utuh.
8. **B. menggeser 50px ke kanan** — translate geser; rotate putar; scale ubah ukuran.
9. **B. membesar 10% (zoom halus)** — scale(1.1) = 110% ukuran — efek kartu populer.
10. **B. mendefinisikan tahapan animasi** — Dipakai bersama properti animation.
11. **B. --warna: red; dipakai var(--warna)** — Custom property biasanya di :root.
12. **A. @media (max-width: 600px){ }** — Kunci desain responsif mobile.
13. **C. rem** — rem = root em.
14. **B. setinggi viewport penuh** — vh = viewport height; vw = lebar.

15. **A. ::before** — ::before/::after + content: ''.

16. **pointer** — pointer menandakan elemen bisa diklik.

17. **rotate** — rotate memutar elemen.

Spesifisitas, Variabel & Responsif Modern

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

1. Siapa yang menang saat aturan bertabrakan?

```
inline style  >  #id  >  .class / [atribut] / :hover  >
tag
kekuatan sama → yang ditulis PALING AKHIR menang
!important    → mengalahkan hampir semuanya (hindari)
```

2. CSS variable — satu sumber warna untuk seluruh situs:

```
:root { --utama: #4a8c7f; --bg: #f4f5ef; }
@media (prefers-color-scheme: dark) {
  :root { --utama: #77b5a7; --bg: #161c1a; }
}
.btn { background: var(--utama); }
```

3. Satuan: `rem` (kelipatan font root, 1rem = 16px), `em` (kelipatan font induk), `%`, `vw/vh` (persen layar), `px` (tetap).

4. Responsif tanpa banyak media query:

```
h1     { font-size: clamp(1.2rem, 4vw, 2rem); }
.kartu { display: grid; grid-template-columns:
repeat(auto-fill, minmax(220px, 1fr)); gap: 14px; }
@media (min-width: 768px) { .sidebar { display: block;
} }   /* mobile-first */
```

5. Aksesibilitas di CSS: jangan hapus outline tanpa pengganti → pakai `:focus-visible`; hormati `prefers-reduced-motion` untuk mematikan animasi.

💡 Buka DevTools (F12) → tab Elements → panel Styles. Aturan yang kalah tampil dicoret — cara tercepat memahami spesifisitas.

🧠 Uji Diri — 11 soal

1. **Elemen <p class="info" id="pesan">. Warna teksnya adalah...**

```
#pesan { color: blue; }
.info  { color: green; }
p      { color: red; }
```

   A. blue
   B. green
   C. red
   D. Campuran ketiganya

2. **Dua aturan dengan spesifisitas SAMA bertabrakan. Yang menang:**

```
.tombol { background: green; }
.tombol { background: orange; }
```

   A. Yang ditulis paling akhir (orange)
   B. Yang ditulis pertama (green)
   C. Campuran warnanya
   D. Tidak ada, error

3. **Kenapa !important sebaiknya dihindari?**

   A. Ia mengalahkan hampir semua aturan lain sehingga perubahan berikutnya ikut butuh !important — CSS jadi sulit dirawat
   B. Karena membuat halaman lambat
   C. Karena tidak didukung browser
   D. Karena hanya untuk warna

4. **Cara memakai CSS variable (custom property) ini:**

```
:root { --utama: #4a8c7f; }
```

   A. .btn { background: var(--utama); }
   B. .btn { background: --utama; }
   C. .btn { background: $utama; }
   D. .btn { background: utama(); }

5. **Jika ukuran font root (html) = 16px, maka 1.5rem = ...**

   A. 24px
   B. 1.5px
   C. 16px
   D. 15px

6. **Pendekatan 'mobile-first' menulis media query seperti ini:**

   A. Gaya dasar untuk HP, lalu @media (min-width: 768px) { ... } untuk layar yang lebih besar
   B. Gaya dasar untuk desktop, lalu @media (max-width: 768px) untuk HP
   C. Membuat dua file HTML terpisah
   D. Tidak memakai media query

7. **font-size: clamp(1.2rem, 4vw, 2rem) artinya...**

   A. Ukuran mengikuti 4% lebar layar, tapi tidak pernah lebih kecil dari 1.2rem dan tidak lebih besar dari 2rem
   B. Selalu 4vw
   C. Selalu 2rem
   D. Font akan berkedip

8. **Grid kartu yang otomatis menyesuaikan jumlah kolom tanpa media query:**

   A. grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
   B. grid-template-columns: 220px 220px 220px;
   C. display: block;
   D. float: left; width: 33%;

9. **Pseudo-class yang tepat untuk menampilkan garis fokus saat user menavigasi dengan KEYBOARD:**

   A. :focus-visible
   B. :hover
   C. :active
   D. :visited

10. **Media query untuk mendeteksi bahwa pengguna memakai mode gelap di sistemnya:**

    A. @media (prefers-color-scheme: dark) { ... }
    B. @media (dark-mode: on) { ... }
    C. @media screen and (night) { ... }
    D. @dark { ... }

11. **Fungsi CSS untuk membaca nilai custom property: background: ___(--utama);**

    Jawab: _____

Kunci & Pembahasan · CSS Bab 7

← Kembali ke soal · Awal bab

1. **A. blue** — Spesifisitas: id (#) > class (.) / atribut / pseudo-class > tag. Selector id paling kuat, jadi biru menang walau ditulis paling atas. Itu sebabnya banyak tim menghindari id untuk styling — terlalu kuat dan sulit ditimpa.
   - **B:** Class kalah dari id.
   - **C:** Selector tag paling lemah.
   - **D:** Hanya satu nilai color yang dipakai.

2. **A. Yang ditulis paling akhir (orange)** — Itulah arti 'cascading' di Cascading Style Sheets: jika kekuatannya sama, aturan yang muncul belakangan menang. Urutan file CSS (link) juga berpengaruh dengan cara yang sama.
   - **B:** Aturan yang lebih awal tertimpa.
   - **C:** CSS tidak mencampur nilai.
   - **D:** Tabrakan aturan adalah hal normal, bukan error.

3. **A. Ia mengalahkan hampir semua aturan lain sehingga perubahan berikutnya ikut butuh !important — CSS jadi sulit dirawat** — !important adalah 'jalan pintas' yang menumpuk menjadi masalah. Lebih baik perbaiki spesifisitas atau urutan aturan. Pengecualian yang wajar: class utilitas seperti .tersembunyi { display: none !important; }.
   - **B:** Dampak performanya tidak berarti.
   - **C:** !important didukung semua browser.
   - **D:** Berlaku untuk properti apa pun.

4. **A. .btn { background: var(--utama); }** — Variabel CSS dideklarasikan dengan awalan -- lalu dibaca dengan var(). Ubah satu nilai di :root, dan seluruh situs ikut berubah. KodeQuiz memakai teknik ini untuk mode gelap: variabel yang sama diberi nilai berbeda.
    - **B:** Harus dibungkus var().
    - **C:** $ adalah sintaks Sass, bukan CSS murni.
    - **D:** Tidak ada fungsi seperti itu.

5. **A. 24px** — rem = kelipatan font-size root: 1.5 × 16 = 24px. Memakai rem membuat seluruh ukuran ikut membesar jika pengguna memperbesar font di pengaturan browser — lebih aksesibel daripada px. em mengikuti font-size elemen induknya.
    - **B:** rem adalah kelipatan, bukan piksel langsung.
    - **C:** Itu 1rem.
    - **D:** Bukan hasil perkalian 1,5 × 16.

6. **A. Gaya dasar untuk HP, lalu @media (min-width: 768px) { ... } untuk layar yang lebih besar** — Mulai dari layar kecil memaksa kamu memprioritaskan konten penting, lalu menambah tata letak saat layar lebar. Desktop-first (max-width) juga bisa, tapi biasanya menghasilkan lebih banyak aturan yang harus dibatalkan.
    - **B:** Itu desktop-first.
    - **C:** Satu HTML yang responsif lebih mudah dirawat.
    - **D:** Tanpa media query, tata letak sulit menyesuaikan layar.

7. **A. Ukuran mengikuti 4% lebar layar, tapi tidak pernah lebih kecil dari 1.2rem dan tidak lebih besar dari 2rem** — clamp(MIN, IDEAL, MAX) membuat tipografi responsif tanpa media query. Judul besar di desktop, tetap terbaca di HP.
   - **B:** 4vw dibatasi oleh nilai MIN dan MAX.
   - **C:** 2rem hanya batas atas.
   - **D:** clamp tidak membuat animasi.

8. **A. grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));** — Setiap kolom minimal 220px dan membagi sisa ruang secara rata (1fr). Di HP jadi 1 kolom, di laptop 3–4 kolom. Beranda KodeQuiz memakai teknik ini untuk kartu topik.
   - **B:** Kolom tetap tidak menyesuaikan lebar layar.
   - **C:** Block tidak membuat kolom.
   - **D:** Float adalah teknik lama yang rapuh.

9. **A. :focus-visible** — Menghapus outline fokus (outline: none) tanpa pengganti membuat pengguna keyboard tersesat. :focus-visible hanya muncul saat navigasi keyboard, sehingga tidak mengganggu pengguna mouse.
   - **B:** :hover untuk mouse.
   - **C:** :active saat sedang ditekan.
   - **D:** :visited untuk link yang pernah dikunjungi.

10. **A. @media (prefers-color-scheme: dark) { ... }** — Kombinasikan dengan CSS variable: ubah nilai --bg, --teks, dll. di dalam media query tersebut, dan seluruh halaman otomatis berganti tema. Ada juga prefers-reduced-motion untuk pengguna yang ingin animasi minimal.
    - **B:** Tidak ada fitur media dark-mode.
    - **C:** Bukan sintaks media query yang valid.
    - **D:** Bukan at-rule yang valid.
11. **var** — var(--nama, nilai_cadangan). Nilai cadangan dipakai jika variabelnya belum didefinisikan.

BAGIAN 3

Java

Bukan prioritas (opsional) — Banyak di lowongan backend & Android, tapi bukan jalur utamamu.

Tipe data, kontrol, array & String, OOP, koleksi

5 bab · 40 soal

Dasar Java: Struktur Program & Tipe Data

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Kenapa bukan prioritas untukmu? Jalur utamamu (Data Analyst, BI, Odoo Consultant) memakai SQL, Python, dan JavaScript. Java tetap berguna: banyak lowongan backend di Indonesia memakainya, dan OOP-nya sangat rapi untuk dipelajari.

```
public class Main {
    public static void main(String[] args) {     //
titik masuk program
        int    umur  = 22;          // bilangan bulat
        double harga = 150000.0;    // desimal
        String nama  = "Rina";      // teks (class, S
besar)
        char   kode  = 'A';         // satu karakter
        boolean aktif = true;
        final double PAJAK = 0.11;  // konstanta
        var total = harga * 3;      // tipe disimpulkan
(Java 10+)

        System.out.println("Halo " + nama);   // cetak
+ pindah baris
    }
}
```

- Java **bertipe statis**: tipe diperiksa sebelum program jalan → banyak bug tertangkap lebih awal.
- Setiap pernyataan diakhiri `;`, blok memakai `{ }`.
- `7 / 2 = 3` (int ÷ int), `7 / 2.0 = 3.5`.
- `"Total: " + 2 + 3 = "Total: 23"` — gunakan kurung `(2 + 3)`.

🧪 **Coba tanpa install:** cari "online Java compiler" di browser, tempel kodenya, lalu tekan Run. Jika JDK sudah ter-install: simpan sebagai `Main.java`, lalu jalankan `java Main.java`.

🧠 Uji Diri — 8 soal

1. **Program Java mulai dijalankan dari method...**

   A. public static void main(String[] args)
   B. start()
   C. run()
   D. init()

2. **Output kode ini:**

```
System.out.println("Halo");
System.out.print("Data");
System.out.print("Analis");
```

   A. Halo DataAnalis
   B. Halo Data Analis
   C. HaloDataAnalis
   D. Halo Data Analis

3. **Kode int umur = "dua puluh"; menghasilkan...**

   A. Error saat kompilasi — Java bertipe statis; variabel int hanya boleh berisi bilangan bulat
   B. umur bernilai 20
   C. umur bernilai 0
   D. Berjalan normal

4. **Output kode ini:**

```
System.out.println(7 / 2);
System.out.println(7 / 2.0);
```

A. 3 3.5
B. 3.5 3.5
C. 3 3
D. 4 3.5

5. **Output kode ini:**

```
System.out.println("Total: " + 2 + 3);
System.out.println(2 + 3 + " item");
```

A. Total: 23 5 item
B. Total: 5 5 item
C. Total: 23 23 item
D. Total: 5 23 item

6. **Tipe data Java untuk menyimpan teks seperti nama pelanggan:**

A. String
B. char
C. int
D. boolean

7. **Kata kunci final pada final double PAJAK = 0.11; artinya...**

A. Nilainya tidak bisa diubah setelah diisi (konstanta)
B. Variabel terakhir di program
C. Variabel akan dihapus
D. Hanya bisa dipakai sekali

8. **Sejak Java 10, variabel lokal bisa dideklarasikan tanpa menulis tipenya dengan kata kunci ___ (contoh: ___ total = 0;)**

   Jawab: _____

Kunci & Pembahasan · Java Bab 1

← Kembali ke soal · Awal bab

1. **A. public static void main(String[] args)** — Setiap program Java (yang berdiri sendiri) membutuhkan method main sebagai titik masuk. Semua kode berada di dalam class — Java sangat berorientasi objek.
   - **B:** start() dipakai di konteks lain (misal Thread), bukan titik masuk program.
   - **C:** run() bukan titik masuk program.
   - **D:** init() bukan titik masuk program Java biasa.

2. **A. Halo DataAnalis** — println mencetak lalu pindah baris; print tidak pindah baris. Jadi 'Data' dan 'Analis' menempel di baris kedua.
   - **B:** Tidak ada spasi yang dicetak di antara kata.
   - **C:** println pertama menambahkan pindah baris setelah 'Halo'.
   - **D:** print tidak menambahkan pindah baris.

3. **A. Error saat kompilasi — Java bertipe statis; variabel int hanya boleh berisi bilangan bulat** — Java (seperti C#, Kotlin, TypeScript) memeriksa tipe sebelum program dijalankan. Banyak kesalahan tertangkap lebih awal — beda dengan Python/JavaScript yang baru ketahuan saat program berjalan.
   - **B:** Java tidak menerjemahkan kata menjadi angka.
   - **C:** Nilai tidak diisi otomatis; kodenya tidak bisa dikompilasi.
   - **D:** Tipe data tidak cocok, jadi tidak bisa berjalan.

4. **A. 3 3.5** — int ÷ int = int (desimal dibuang, bukan dibulatkan). Jika salah satu bilangan desimal (2.0), hasilnya double. Jebakan yang sama ada di SQL (SQLite/PostgreSQL).
   - **B:** 7 / 2 dengan dua int menghasilkan int.
   - **C:** 7 / 2.0 menghasilkan double karena 2.0 desimal.
   - **D:** Pembagian int membuang desimal, tidak membulatkan ke atas.

5. **A. Total: 23 5 item** — Dikerjakan dari kiri. Baris 1: teks + 2 = teks, lalu + 3 ditempel → 'Total: 23'. Baris 2: 2 + 3 = 5 dulu (angka), baru digabung dengan teks. Sama persis dengan JavaScript. Pakai kurung: "Total: " + (2 + 3).
   - **B:** Setelah bertemu teks, + menjadi penggabungan.
   - **C:** Di baris 2, angka dijumlah dulu sebelum bertemu teks.
   - **D:** Terbalik.

6. **A. String** — String (huruf S kapital karena ia sebuah class). char hanya menyimpan SATU karakter dengan kutip tunggal: 'A'. Tipe primitif lain: int, long, double, boolean.
   - **B:** char hanya satu karakter.
   - **C:** int untuk bilangan bulat.
   - **D:** boolean untuk true/false.

7. **A. Nilainya tidak bisa diubah setelah diisi (konstanta)** — final = konstanta. Konvensi penamaannya HURUF_BESAR. Mirip const di JavaScript.
   - **B:** Bukan soal urutan.
   - **C:** final tidak menghapus apa pun.
   - **D:** Konstanta bisa dibaca berkali-kali.

8. **var** — var total = 0; → Java menyimpulkan tipenya int. Tipenya tetap statis; hanya penulisannya yang lebih ringkas.

Kontrol Alur: if, switch, for, while

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
if (nilai >= 80) { … } else if (nilai >= 70) { … } else
{ … }

String jenis = switch (hari) {              // Java 14+
    case "Sabtu", "Minggu" -> "Libur";
    default -> "Kerja";
};

String label = qty >= 5 ? "Grosir" : "Ecer";   //
ternary

for (int i = 1; i <= 5; i++) { … }
while (saldo < 150) { … }                       //
break / continue
```

⚠️ **Jebakan Java paling terkenal:** bandingkan teks dengan `a.equals(b)`, bukan `a == b`. `==` pada objek membandingkan alamatnya, bukan isinya.

🧠 Uji Diri — 8 soal

1. **Output kode ini:**

```
int nilai = 75;
if (nilai >= 80) {
    System.out.println("A");
} else if (nilai >= 70) {
    System.out.println("B");
} else {
    System.out.println("C");
}
```

A. B

B. A

C. C

D. B C

2. **Output kode ini:**

```
int total = 0;
for (int i = 1; i <= 5; i++) {
    total += i;
}
System.out.println(total);
```

A. 15

B. 10

C. 5

D. 14

3. **Output kode ini (Java 14+):**

```
String hari = "Sabtu";
String jenis = switch (hari) {
    case "Sabtu", "Minggu" -> "Libur";
    default -> "Kerja";
};
System.out.println(jenis);
```

A. Libur
B. Kerja
C. Sabtu
D. Error

4. **Output kode ini:**

```
int qty = 5;
System.out.println(qty >= 5 ? "Grosir" : "Ecer");
```

A. Grosir
B. Ecer
C. true
D. 5

5. **Output kode ini:**

```
String a = new String("Andi");
String b = "Andi";
System.out.println(a == b);
System.out.println(a.equals(b));
```

A. false true
B. true true
C. false false
D. true false

6. **Output kode ini:**

```
int n = 0;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        n++;
    }
}
System.out.println(n);
```

A. 6
B. 5
C. 3
D. 2

7. **Output kode ini:**

```
int saldo = 100;
int bulan = 0;
while (true) {
    saldo += 20;
    bulan++;
    if (saldo >= 150) break;
}
System.out.println(bulan);
```

A. 3
B. 2
C. 50
D. Loop tak berhenti

8. **Kata kunci untuk keluar dari loop seketika: _____**

Jawab: _____

Kunci & Pembahasan · Java Bab 2

← Kembali ke soal · Awal bab

1. **A. B** — Syarat diperiksa berurutan; 75 tidak ≥ 80, tapi ≥ 70 → B. Setelah satu cabang dijalankan, sisanya dilewati. Logikanya sama dengan IF bertingkat Excel dan CASE WHEN SQL.
   - **B:** 75 belum mencapai 80.
   - **C:** Cabang else hanya jika semua syarat salah.
   - **D:** Hanya satu cabang yang dijalankan.
2. **A. 15** — for (awal; syarat; langkah). i = 1..5 (karena <=), dijumlah: 1+2+3+4+5 = 15.
   - **B:** 10 jika syaratnya i < 5.
   - **C:** 5 hanya nilai terakhir i.
   - **D:** Tidak ada angka yang terlewat.
3. **A. Libur** — Switch expression modern dengan panah (→) mengembalikan nilai dan tidak 'jatuh' ke case berikutnya. Satu case boleh berisi beberapa nilai yang dipisah koma.
   - **B:** default hanya jika tidak ada case yang cocok.
   - **C:** switch mengembalikan nilai case, bukan isi variabelnya.
   - **D:** Sintaks ini valid sejak Java 14.
4. **A. Grosir** — Operator ternary: syarat ? nilai_jika_benar : nilai_jika_salah. Ringkas untuk pilihan sederhana; untuk logika panjang lebih jelas memakai if-else.
   - **B:** 5 >= 5 bernilai benar.
   - **C:** Yang dicetak nilai pilihan, bukan hasil syaratnya.
   - **D:** Ternary mengembalikan salah satu teks.

5. **A. false true** — Untuk objek (termasuk String), == membandingkan REFERENSI (apakah objek yang sama), equals() membandingkan ISI. Selalu pakai .equals() untuk membandingkan teks di Java. Bug ini sangat sering ditanyakan di tes Java.
    - **B:** new String membuat objek baru, jadi == bernilai false.
    - **C:** equals membandingkan isi teks yang sama.
    - **D:** Terbalik.
6. **A. 6** — Loop luar 3 kali × loop dalam 2 kali = 6. Pola ini muncul saat memproses tabel (baris × kolom).
    - **B:** Hitung ulang: 3 × 2.
    - **C:** 3 hanya jumlah putaran luar.
    - **D:** 2 hanya jumlah putaran dalam.
7. **A. 3** — 100 → 120 → 140 → 160. Di bulan ke-3 saldo ≥ 150, lalu break menghentikan loop. while (true) + break berguna saat syarat berhenti baru diketahui di tengah perulangan.
    - **B:** Setelah 2 bulan saldo baru 140.
    - **C:** 50 adalah selisih yang dicari, bukan jumlah bulan.
    - **D:** break menghentikan loop.
8. **break** — break keluar dari loop; continue melompat ke putaran berikutnya. Keduanya juga ada di Python, JavaScript, dan PHP.

Array & String

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
int[] angka = {3, 8, 2};
angka[0];          // 3 (indeks mulai 0)
angka.length;      // 3  ← properti, tanpa ()
Arrays.sort(angka);
Arrays.toString(angka);   // "[2, 3, 8]"

for (int a : angka) { … }  // for-each

String kota = "Jakarta";
kota.length();             // 7  ← method, pakai ()
kota.substring(0, 4);      // "Jaka"
kota.charAt(0);            // 'J'
kota.toUpperCase();        // "JAKARTA" (string baru!)
"a,b,c".split(",");        // ["a", "b", "c"]
```

String **immutable**: method seperti `toUpperCase()` menghasilkan string baru, jadi simpan hasilnya: `s = s.toUpperCase();`.

🧠 Uji Diri — 8 soal

1. **Output kode ini:**

```
int[] angka = {3, 8, 2};
System.out.println(angka[1]);
System.out.println(angka.length);
```

A. 8 3
B. 3 3
C. 8 2
D. 3 2

2. **Mengakses angka[3] pada array int[] angka = {3, 8, 2} menghasilkan...**

   A. ArrayIndexOutOfBoundsException
   B. 0
   C. null
   D. 2

3. **Output kode ini:**

```
String kota = "Jakarta";
System.out.println(kota.substring(0, 4));
System.out.println(kota.charAt(0));
```

   A. Jaka J
   B. Jakar J
   C. Jaka a
   D. akar J

4. **Output kode ini:**

```
int[] total = {14, 2, 6};
int jumlah = 0;
for (int t : total) {
    jumlah += t;
}
System.out.println(jumlah);
```

   A. 22
   B. 3
   C. 14
   D. 146

5. **Output kode ini:**

```
String s = "data";
s.toUpperCase();
System.out.println(s);
```

A. data
B. DATA
C. Data
D. Error

6. **Output kode ini:**

```
int[] a = {5, 1, 4};
Arrays.sort(a);
System.out.println(Arrays.toString(a));
```

A. [1, 4, 5]
B. [5, 4, 1]
C. [5, 1, 4]
D. 1 4 5

7. **Output kode ini:**

```
String[] bagian =
"Andi,Jakarta,Laptop".split(",");
System.out.println(bagian.length + " " +
bagian[2]);
```

A. 3 Laptop
B. 3 Jakarta
C. 2 Laptop
D. 19 Laptop

8. **Method String untuk membandingkan ISI teks di Java: a._____(b)**

   Jawab: _____

Kunci & Pembahasan · Java Bab 3

← Kembali ke soal · Awal bab

1. **A. 8 3** — Indeks dimulai dari 0, jadi angka[1] = 8. Panjang array adalah PROPERTI length (tanpa kurung), sedangkan panjang String adalah METHOD length() (dengan kurung).
   - **B:** Indeks 1 adalah elemen kedua.
   - **C:** Ada 3 elemen di array.
   - **D:** Keduanya keliru.

2. **A. ArrayIndexOutOfBoundsException** — Indeks yang valid hanya 0 sampai length − 1 (0, 1, 2). Java melempar exception, bukan mengembalikan nilai kosong seperti JavaScript (undefined).
   - **B:** Java tidak mengembalikan nilai default untuk indeks di luar batas.
   - **C:** Array int tidak berisi null.
   - **D:** 2 adalah isi indeks terakhir yang valid.

3. **A. Jaka J** — substring(awal, akhir): mulai dari indeks awal sampai SEBELUM indeks akhir → indeks 0–3 → 'Jaka'. charAt(0) = karakter pertama 'J'. Sama dengan slicing Python [0:4].
   - **B:** Indeks akhir (4) tidak ikut diambil.
   - **C:** charAt(0) mengambil karakter pertama.
   - **D:** substring dimulai dari indeks 0.

4. **A. 22** — Enhanced for (for-each) mengambil setiap elemen tanpa perlu indeks — mirip for t in total di Python. 14 + 2 + 6 = 22.
   - **B:** 3 adalah panjang array.
   - **C:** Itu hanya elemen pertama.
   - **D:** Angka dijumlah, bukan digabung sebagai teks.

5. **A. data** — String di Java bersifat immutable (tidak bisa diubah). toUpperCase() menghasilkan String BARU, tapi hasilnya tidak disimpan. Yang benar: s = s.toUpperCase(); — sama seperti string di Python dan JavaScript.
    - **B:** Hasil toUpperCase() tidak disimpan ke s.
    - **C:** Tidak ada perubahan pada s.
    - **D:** Kode ini valid.
6. **A. [1, 4, 5]** — Arrays.sort mengurutkan naik (langsung mengubah array-nya). Arrays.toString menampilkan isi array dengan rapi. Mencetak array langsung (println(a)) hanya menampilkan alamat seperti [I@1b6d3586.
    - **B:** sort mengurutkan naik, bukan turun.
    - **C:** Array sudah diurutkan sebelum dicetak.
    - **D:** toString menampilkan format dengan kurung siku dan koma.
7. **A. 3 Laptop** — split memecah teks berdasarkan pemisah menjadi array: ["Andi", "Jakarta", "Laptop"]. Dipakai saat membaca baris CSV — seperti Text to Columns di Excel.
    - **B:** Indeks 2 adalah elemen ketiga.
    - **C:** Ada 3 bagian setelah dipisah koma.
    - **D:** 19 adalah panjang teks aslinya.
8. **equals** — a.equals(b) membandingkan isi. equalsIgnoreCase(b) membandingkan tanpa memedulikan huruf besar/kecil.

OOP: Class, Object, Inheritance, Interface

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
class Produk {
    private String nama;          // encapsulation:
field private
    private int harga;

    Produk(String nama, int harga) {   // constructor
        this.nama = nama;
        this.harga = harga;
    }
    int getHarga() { return harga; }   // getter
    int totalHarga(int qty) { return harga * qty; }
}

class Karyawan { double gaji() { return 5_000_000; } }
class Sales extends Karyawan {          // inheritance
    @Override double gaji() { return super.gaji() +
1_000_000; }
}

interface BisaDiekspor { String keCsv(); }   // kontrak
class Laporan implements BisaDiekspor { public String
keCsv() { return "…"; } }
```

Empat pilar OOP: encapsulation (lindungi data), inheritance (warisi), polymorphism (satu tipe banyak perilaku), abstraction (sembunyikan detail).

> 🔗 Konsep yang sama ada di Python (`class Sales(Karyawan)`) dan di Odoo (`_inherit`). Belajar OOP di Java membantu memahami keduanya.

🧠 Uji Diri — 8 soal

1. **Beda class dan object:**

   A. Class = cetakan/rancangan; object = hasil nyata yang dibuat dari class dengan new
   B. Keduanya sama
   C. Object = cetakan; class = hasilnya
   D. Class hanya untuk angka

2. **Output kode ini:**

```
class Produk {
    String nama;
    int harga;

    Produk(String nama, int harga) {
        this.nama = nama;
        this.harga = harga;
    }

    int totalHarga(int qty) {
        return harga * qty;
    }
}

public class Main {
    public static void main(String[] args) {
        Produk p = new Produk("Mouse", 150000);
        System.out.println(p.totalHarga(3));
    }
}
```

   A. 450000
   B. 150000
   C. 3
   D. Mouse

3. **Prinsip encapsulation di Java biasanya diterapkan dengan...**

   A. Field dibuat private, lalu diakses lewat method public (getter/setter) yang bisa memvalidasi nilai
   B. Semua field dibuat public
   C. Menghapus semua method
   D. Memakai satu class untuk semua hal

4. **Output kode ini:**

```
class Karyawan {
    double gaji() { return 5000000; }
}

class Sales extends Karyawan {
    double komisi = 1000000;

    @Override
    double gaji() { return super.gaji() + komisi; }
}

public class Main {
    public static void main(String[] args) {
        Karyawan k = new Sales();
        System.out.println(k.gaji());
    }
}
```

   A. 6000000.0
   B. 5000000.0
   C. 1000000.0
   D. 6000000

5. **interface di Java berfungsi sebagai...**

```
interface BisaDiekspor {
    String keCsv();
}

class Laporan implements BisaDiekspor {
    public String keCsv() { return "a,b,c"; }
}
```

   A. Kontrak: class yang implements wajib menyediakan semua method yang dideklarasikan
   B. Class yang bisa langsung dibuat object-nya dengan new
   C. Tempat menyimpan data
   D. Pengganti method main

6. **Anggota class yang ditandai static...**

   A. Milik class itu sendiri, bukan milik tiap object — dipanggil dengan NamaClass.anggota
   B. Tidak bisa diubah
   C. Hanya untuk teks
   D. Otomatis dihapus

7. **Urutan access modifier dari yang paling tertutup ke paling terbuka:**

   A. private → (default/package) → protected → public
   B. public → protected → private
   C. protected → private → public
   D. Semua sama

8. **Kata kunci untuk mewarisi class lain: class Sales _____ Karyawan**

   Jawab: _____

Kunci & Pembahasan · Java Bab 4

← Kembali ke soal · Awal bab

1. **A. Class = cetakan/rancangan; object = hasil nyata yang dibuat dari class dengan new** — class Produk mendefinisikan field (nama, harga) dan method. new Produk("Mouse", 150000) membuat satu object nyata. Konsep yang sama ada di Python (class) dan model Odoo.
   - **B:** Perannya berbeda.
   - **C:** Terbalik.
   - **D:** Class bisa memodelkan apa pun.

2. **A. 450000** — Constructor mengisi field object; this.nama menunjuk field milik object, bukan parameter yang namanya sama. totalHarga(3) = 150.000 × 3 = 450.000.
   - **B:** Itu harga satuannya.
   - **C:** 3 adalah qty yang dikirim.
   - **D:** Method mengembalikan angka, bukan nama.

3. **A. Field dibuat private, lalu diakses lewat method public (getter/setter) yang bisa memvalidasi nilai** — Contoh: setHarga(int h) { if (h < 0) throw ...; this.harga = h; } — harga negatif tidak bisa masuk. Data dilindungi dari perubahan sembarangan.
   - **B:** Field public bisa diubah tanpa validasi.
   - **C:** Method justru menjadi pintu akses yang aman.
   - **D:** Itu bertentangan dengan desain OOP yang baik.

4. **A. 6000000.0** — Inheritance (extends) + override + polymorphism: variabel bertipe Karyawan berisi object Sales, jadi yang dipanggil versi Sales. super.gaji() memanggil versi induknya (5 jt), ditambah komisi = 6 jt. double dicetak dengan .0.
    - **B:** Method yang dipanggil adalah versi Sales (object sebenarnya).
    - **C:** Itu hanya komisinya.
    - **D:** Tipe kembaliannya double, jadi dicetak 6000000.0.
5. **A. Kontrak: class yang implements wajib menyediakan semua method yang dideklarasikan** — Interface menjamin kemampuan tertentu. Satu class bisa mengimplementasikan banyak interface, tapi hanya bisa extends satu class induk.
    - **B:** Interface tidak bisa langsung dibuat object-nya.
    - **C:** Interface tidak untuk menyimpan data object.
    - **D:** main tetap diperlukan sebagai titik masuk.
6. **A. Milik class itu sendiri, bukan milik tiap object — dipanggil dengan NamaClass.anggota** — Contoh: Math.max(3, 8) dipanggil tanpa new Math(). Penghitung jumlah object yang sudah dibuat juga biasanya static. Bedakan dengan final (tidak bisa diubah).
    - **B:** Tidak bisa diubah adalah arti final.
    - **C:** static bisa untuk tipe apa pun.
    - **D:** static tidak terkait penghapusan.

7. **A. private → (default/package) → protected → public** — private: hanya di class itu. Default (tanpa kata kunci): satu package. protected: package + subclass. public: semua. Aturan praktis: mulai dari private, buka seperlunya.
    - **B:** Itu dari yang paling terbuka.
    - **C:** Urutannya keliru.
    - **D:** Tingkat aksesnya berbeda-beda.
8. **extends** — extends untuk class (hanya satu induk), implements untuk interface (boleh banyak). Di Python: class Sales(Karyawan); di Odoo: _inherit.

Koleksi (ArrayList, HashMap), Exception & Stream

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
import java.util.*;

ArrayList<String> sales = new ArrayList<>();
sales.add("Andi"); sales.remove("Andi"); sales.size();
sales.get(0);

HashMap<String, Integer> stok = new HashMap<>();
stok.put("Laptop", 5);                          // kunci
unik → put menimpa
stok.getOrDefault("Mouse", 0);

try {
    int x = Integer.parseInt("12a");
} catch (NumberFormatException e) {
    System.out.println("Bukan angka");
}

int hasil = List.of(14, 2, 6, 7).stream()
    .filter(t -> t >=
6).mapToInt(Integer::intValue).sum();   // 27
```

Generics memakai class pembungkus: `ArrayList<Integer>`, bukan `<int>`.

> 🧭 Java populer untuk backend (Spring Boot), perbankan, dan Android. Pelajari secukupnya agar bisa membaca kode dan lolos tes dasar; fokus utamamu tetap SQL, Python, dan JavaScript.

🧠 Uji Diri — 8 soal

1. **Kelebihan ArrayList dibanding array biasa (int[]):**

   A. Ukurannya bisa bertambah/berkurang otomatis dan punya method praktis (add, remove, contains)
   B. Lebih cepat dalam semua kasus
   C. Bisa menyimpan tipe primitif int langsung
   D. Tidak perlu import

2. **Output kode ini:**

```
ArrayList<String> sales = new ArrayList<>();
sales.add("Andi");
sales.add("Bima");
sales.add("Citra");
sales.remove("Bima");
System.out.println(sales.size() + " " +
sales.get(1));
```

   A. 2 Citra
   B. 3 Bima
   C. 2 Bima
   D. 2 Andi

3. **Output kode ini:**

```
HashMap<String, Integer> stok = new HashMap<>();
stok.put("Laptop", 5);
stok.put("Mouse", 20);
stok.put("Laptop", 3);
System.out.println(stok.get("Laptop") + " " +
stok.size());
```

A. 3 2

B. 5 2

C. 8 2

D. 3 3

4. **Output kode ini:**

```
String[] kota = {"Jakarta", "Bandung", "Jakarta"};
HashMap<String, Integer> hitung = new HashMap<>();
for (String k : kota) {
    hitung.put(k, hitung.getOrDefault(k, 0) + 1);
}
System.out.println(hitung.get("Jakarta"));
```

A. 2

B. 1

C. 3

D. null

5. **Output kode ini:**

```
try {
    int x = Integer.parseInt("12a");
    System.out.println(x);
} catch (NumberFormatException e) {
    System.out.println("Bukan angka");
}
```

A. Bukan angka
B. 12
C. 0
D. Program berhenti dengan error

6. **Kenapa ArrayList<int> tidak bisa dipakai, sedangkan ArrayList<Integer> bisa?**

A. Generics hanya menerima tipe objek; int adalah tipe primitif, jadi dipakai pembungkusnya Integer
B. int terlalu kecil
C. Integer lebih cepat dari int
D. Itu hanya aturan penulisan, keduanya sama

7. **Output kode ini:**

```
List<Integer> total = List.of(14, 2, 6, 7);
int hasil = total.stream()
                 .filter(t -> t >= 6)
                 .mapToInt(Integer::intValue)
                 .sum();
System.out.println(hasil);
```

A. 27
B. 29
C. 4
D. 21

8. **Di mana Java paling banyak dipakai di dunia kerja?**

   A. Backend perusahaan besar (misal Spring Boot), sistem perbankan, dan aplikasi Android (bersama Kotlin)
   B. Mendesain logo
   C. Membuat spreadsheet
   D. Hanya untuk tugas kuliah

Kunci & Pembahasan · Java Bab 5

← Kembali ke soal · Awal bab

1. **A. Ukurannya bisa bertambah/berkurang otomatis dan punya method praktis (add, remove, contains)** — Array ukurannya tetap sejak dibuat. ArrayList tumbuh sesuai kebutuhan — mirip list di Python dan array di JavaScript.
   - **B:** Array biasa kadang lebih cepat.
   - **C:** ArrayList memakai class pembungkus (Integer), bukan int.
   - **D:** Perlu import java.util.ArrayList.

2. **A. 2 Citra** — Setelah Bima dihapus, isinya [Andi, Citra] dan elemen di belakangnya bergeser maju. size() = 2, get(1) = Citra.
   - **B:** Bima sudah dihapus.
   - **C:** Elemen bergeser setelah penghapusan.
   - **D:** get(1) adalah elemen kedua.

3. **A. 3 2** — HashMap menyimpan pasangan kunci → nilai (seperti dict Python). Kunci bersifat unik: put dengan kunci yang sama MENIMPA nilai lama. Jadi Laptop = 3 dan jumlah kunci tetap 2.
   - **B:** Nilai 5 sudah ditimpa.
   - **C:** put tidak menjumlahkan nilai.
   - **D:** Kunci Laptop tidak bertambah, hanya diperbarui.

4. **A. 2** — Pola menghitung frekuensi — versi Java dari d.get(k, 0) + 1 di Python. getOrDefault mengembalikan 0 jika kunci belum ada.
   - **B:** Jakarta muncul dua kali.
   - **C:** Bandung tidak ikut dihitung sebagai Jakarta.
   - **D:** Kunci Jakarta sudah diisi.

5. **A. Bukan angka** — parseInt gagal → melempar NumberFormatException → ditangkap catch. Baris println(x) tidak pernah dijalankan. Sama dengan try/except di Python.
   - **B:** parseInt tidak mengambil angka di depan secara otomatis.
   - **C:** Tidak ada nilai default; masuk ke catch.
   - **D:** Exception-nya ditangkap, jadi program tetap berjalan.
6. **A. Generics hanya menerima tipe objek; int adalah tipe primitif, jadi dipakai pembungkusnya Integer** — Setiap tipe primitif punya class pembungkus: int → Integer, double → Double, boolean → Boolean. Java mengubah keduanya otomatis (autoboxing).
   - **B:** Bukan soal ukuran.
   - **C:** Integer justru sedikit lebih berat dari int.
   - **D:** ArrayList<int> tidak bisa dikompilasi.
7. **A. 27** — Stream API (Java 8+) mirip rantai filter-map-reduce di JavaScript atau pandas: ambil yang ≥ 6 (14, 6, 7) lalu jumlahkan = 27.
   - **B:** Itu jumlah semua angka.
   - **C:** 4 adalah banyaknya elemen.
   - **D:** Angka 14 juga lolos filter.

8. **A. Backend perusahaan besar (misal Spring Boot), sistem perbankan, dan aplikasi Android (bersama Kotlin)** — Java banyak muncul di lowongan backend developer, terutama di bank, telko, dan perusahaan besar. Untuk jalur utamamu (Data/BI/Odoo), Java bukan prioritas — tapi konsep OOP-nya membantu memahami bahasa lain.
    - **B:** Desain grafis tidak memakai Java.
    - **C:** Spreadsheet memakai Excel/Sheets.
    - **D:** Java sangat banyak dipakai di industri.

BAGIAN 4

PHP

Bukan prioritas (opsional) — Banyak di lowongan web (Laravel/WordPress). Rencana cadangan.

Dasar, array, form & keamanan web, OOP, Laravel

5 bab · 38 soal

Dasar PHP: Variabel, Teks & Cara Kerjanya

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Kenapa bukan prioritas untukmu? PHP sangat populer di lowongan web developer Indonesia (Laravel, WordPress), tapi jalur utamamu adalah Data/BI/Odoo. Pelajari secukupnya sebagai rencana cadangan dan agar bisa membaca kode web.

PHP berjalan di **server**: browser hanya menerima HTML hasilnya.

```
<?php
$nama  = "Rina";              // variabel selalu diawali
$
$total = 150000;
echo "Halo $nama";            // kutip ganda → variabel
diganti nilainya
echo 'Halo $nama';            // kutip tunggal → apa
adanya
echo "Total: " . $total;      // titik = sambung teks
echo "10" + 5;                // 15 (+ khusus angka)
var_dump("5" === 5);          // bool(false) → pakai ===
```

🧪 **Coba:** jika PHP ter-install, simpan sebagai `coba.php` lalu jalankan `php coba.php` di terminal. Atau gunakan XAMPP/Laragon untuk menjalankan PHP di browser.

🧠 Uji Diri — 7 soal

1. **Di PHP, semua nama variabel diawali tanda...**

   A. $
   B. @
   C. #
   D. &

2. **Output kode ini:**

```
$depan = "Data";
$belakang = "Analis";
echo $depan . " " . $belakang;
```

   A. Data Analis
   B. DataAnalis
   C. Data + Analis
   D. Error

3. **Output kode ini:**

```
$nama = "Rina";
echo "Halo $nama";
echo "\n";
echo 'Halo $nama';
```

   A. Halo Rina Halo $nama
   B. Halo Rina Halo Rina
   C. Halo $nama Halo $nama
   D. Halo $nama Halo Rina

4. **Output kode ini:**

```
echo "10" + 5;
```

A. 15
B. 105
C. Error
D. "10"5

5. **Output kode ini:**

```
var_dump("5" == 5);
var_dump("5" === 5);
```

A. bool(true) bool(false)
B. bool(true) bool(true)
C. bool(false) bool(false)
D. bool(false) bool(true)

6. **Di mana kode PHP dijalankan?**

A. Di SERVER — hasilnya (biasanya HTML) baru dikirim ke browser
B. Di browser pengguna, seperti JavaScript
C. Di Excel
D. Di printer

7. **Setiap pernyataan PHP diakhiri tanda ___**

Jawab: _____

Kunci & Pembahasan · PHP Bab 1

← Kembali ke soal · Awal bab

1. **A. $** — $nama = "Rina"; $total = 150000; — PHP bertipe dinamis seperti Python dan JavaScript: tipe ditentukan dari nilainya.
   - **B:** @ dipakai untuk menekan pesan error (sebaiknya dihindari).
   - **C:** # adalah awal komentar satu baris.
   - **D:** & dipakai untuk referensi.

2. **A. Data Analis** — Di PHP, operator penyambung teks adalah TITIK (.), bukan + seperti di JavaScript/Java. Spasi " " ditambahkan di tengah.
   - **B:** Ada teks spasi yang disambung di tengah.
   - **C:** Tanda + tidak dipakai di kode ini.
   - **D:** Kode ini valid.

3. **A. Halo Rina Halo $nama** — Kutip GANDA mengganti variabel dengan nilainya (interpolasi) dan memproses \n. Kutip TUNGGAL menampilkan teks apa adanya. Mirip template literal `${nama}` di JavaScript dan f-string di Python.
   - **B:** Kutip tunggal tidak menginterpolasi variabel.
   - **C:** Kutip ganda menginterpolasi variabel.
   - **D:** Terbalik.

4. **A. 15** — Karena + di PHP KHUSUS untuk angka, string "10" dikonversi menjadi 10 → 15. Bandingkan dengan JavaScript, di mana "10" + 5 = "105". Penyambung teks PHP adalah titik.
   - **B:** Itu hasil di JavaScript, atau "10" . 5 di PHP.
   - **C:** PHP mengonversi string angka secara otomatis.
   - **D:** Bukan penggabungan teks.

5. **A. bool(true) bool(false)** — Sama seperti JavaScript: == membandingkan setelah konversi tipe, === membandingkan nilai DAN tipe. var_dump menampilkan tipe dan nilainya — berguna untuk debugging.
   - **B:** === tidak mengonversi tipe.
   - **C:** == mengonversi string ke angka.
   - **D:** Terbalik.
6. **A. Di SERVER — hasilnya (biasanya HTML) baru dikirim ke browser** — Pengguna tidak pernah melihat kode PHP-mu, hanya HTML hasilnya. Karena itu PHP bisa mengakses database dengan aman. JavaScript di browser berjalan di sisi pengguna.
   - **B:** JavaScript (frontend) yang berjalan di browser.
   - **C:** Excel tidak menjalankan PHP.
   - **D:** Printer tidak menjalankan kode.
7. **;** — Titik koma. Lupa titik koma adalah penyebab error 'syntax error, unexpected ...' yang paling sering.

Kontrol Alur & Array

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
$kota  = ["Jakarta", "Bandung"];              // array
berindeks
$harga = ["Laptop" => 7000000, "Mouse" => 150000];   //
asosiatif
$kota[] = "Surabaya";                         //
tambah di akhir
count($kota);                                 // 3

foreach ($harga as $produk => $h) {
    echo "$produk: $h\n";
}

if ($nilai >= 80) { … } elseif ($nilai >= 70) { … }
else { … }

echo match (true) {                           // PHP 8
    $nilai >= 90 => "A",
    $nilai >= 80 => "B",
    default      => "C",
};
```

🧠 Uji Diri — 7 soal

1. **Output kode ini:**

```
$kota = ["Jakarta", "Bandung", "Surabaya"];
echo $kota[1] . " " . count($kota);
```

A. Bandung 3
B. Jakarta 3
C. Bandung 2
D. Surabaya 3

2. **Output kode ini:**

```
$harga = ["Laptop" => 7000000, "Mouse" => 150000];
echo $harga["Mouse"];
```

A. 150000
B. 7000000
C. Mouse
D. 1

3. **Output kode ini:**

```
$stok = ["Laptop" => 5, "Mouse" => 20];
foreach ($stok as $produk => $jumlah) {
    echo "$produk: $jumlah\n";
}
```

A. Laptop: 5 Mouse: 20
B. 5: Laptop 20: Mouse
C. Laptop Mouse
D. Laptop: 25

4. **Output kode ini (PHP 8):**

```
$nilai = 85;
echo match (true) {
    $nilai >= 90 => "A",
    $nilai >= 80 => "B",
    default => "C",
};
```

A. B
B. A
C. C
D. true

5. **Output kode ini:**

```
$total = 0;
for ($i = 1; $i <= 4; $i++) {
    $total += $i;
}
echo $total;
```

A. 10
B. 4
C. 6
D. 15

6. **Output kode ini:**

```
$data = [];
$data[] = "Andi";
$data[] = "Bima";
echo count($data) . " " . $data[0];
```

A. 2 Andi
B. 2 Bima
C. 1 Andi
D. 0

7. **Perulangan PHP khusus untuk array: _____ ($data as $baris)**

Jawab: _____

Kunci & Pembahasan · PHP Bab 2

← Kembali ke soal · Awal bab

1. **A. Bandung 3** — Indeks dimulai dari 0, jadi $kota[1] = Bandung. count() menghitung jumlah elemen.
   - **B:** Indeks 1 adalah elemen kedua.
   - **C:** Ada 3 elemen.
   - **D:** Indeks 1 adalah elemen kedua.
2. **A. 150000** — Associative array = pasangan kunci → nilai (seperti dict Python atau objek JavaScript). Sangat sering dipakai untuk data baris hasil query database.
   - **B:** 7000000 adalah harga Laptop.
   - **C:** Yang diambil nilainya, bukan kuncinya.
   - **D:** Kunci 'Mouse' berisi 150000.
3. **A. Laptop: 5 Mouse: 20** — foreach ($array as $kunci ⇒ $nilai) mengulang setiap pasangan. Pola ini dipakai untuk membuat baris tabel HTML dari data.
   - **B:** Urutan kunci dan nilai tertukar.
   - **C:** Nilainya juga dicetak.
   - **D:** Tidak ada penjumlahan di loop ini.
4. **A. B** — match (PHP 8) mengembalikan nilai cabang pertama yang cocok — mirip switch expression di Java dan CASE WHEN di SQL. match(true) dengan syarat di tiap cabang menjadi IF bertingkat yang ringkas.
   - **B:** 85 belum mencapai 90.
   - **C:** default hanya jika tidak ada yang cocok.
   - **D:** match mengembalikan nilai cabang, bukan true.

5. **A. 10** — 1 + 2 + 3 + 4 = 10. Sintaks for PHP sama dengan Java dan JavaScript, hanya variabelnya memakai $.
   - **B:** 4 adalah nilai terakhir $i.
   - **C:** 6 jika syaratnya $i < 4.
   - **D:** 15 jika sampai 5.
6. **A. 2 Andi** — $data[] = nilai menambahkan elemen di akhir array (seperti append/push). Elemen pertama berindeks 0.
   - **B:** Indeks 0 adalah yang ditambahkan pertama.
   - **C:** Ada dua elemen yang ditambahkan.
   - **D:** Array tidak lagi kosong.
7. **foreach** — foreach ($data as $baris) atau foreach ($data as $kunci ⇒ $nilai).

Fungsi & Fungsi Array Bawaan

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
function luas($p, $l = 1) { return $p * $l; }

array_sum([14, 2, 6]);                          // 22
array_filter($data, fn($t) => $t >= 6);         // WHERE
array_map(fn($n) => ucfirst($n), $nama);        // ubah
tiap elemen
implode(", ", $arr);  explode(",", "a,b,c");   //
gabung / pecah teks
arsort($nilai);  array_key_first($nilai);       // urut
turun, ambil teratas
number_format(1234567, 0, ",", ".");            //
"1.234.567"
$_GET["q"] ?? "";                               // nilai
default tanpa warning
```

🔗 Padanan: `array_filter` ≈ filter (JS) ≈ list comprehension (Python) ≈ WHERE (SQL). Satu konsep, banyak bahasa.

🧠 Uji Diri — 8 soal

1. **Output kode ini:**

```
function luas($p, $l = 1) {
    return $p * $l;
}
echo luas(5) . " " . luas(5, 3);
```

A. 5 15

B. 5 5

C. 15 15

D. Error

2. **Output kode ini:**

```
echo array_sum([14, 2, 6]) . " " . max([14, 2, 6]);
```

A. 22 14
B. 22 6
C. 3 14
D. 1426 14

3. **Output kode ini:**

```
$total = [14, 1.5, 6, 0.75, 7];
$besar = array_filter($total, fn($t) => $t >= 6);
echo count($besar);
```

A. 3
B. 2
C. 5
D. 22

4. **Output kode ini:**

```
$nama = ["rina", "tono"];
$rapi = array_map(fn($n) => ucfirst($n), $nama);
echo implode(", ", $rapi);
```

A. Rina, Tono
B. rina, tono
C. RINA, TONO
D. Rina,Tono

5. **Output kode ini:**

```
echo strlen("Keyboard") . " " .
strtoupper("jakarta");
```

A. 8 JAKARTA
B. 8 Jakarta
C. 9 JAKARTA
D. 7 JAKARTA

6. **Output kode ini:**

```
echo number_format(1234567, 0, ",", ".");
```

A. 1.234.567
B. 1,234,567
C. 1234567
D. Rp1.234.567

7. **Output kode ini:**

```
$nilai = ["Andi" => 18, "Bima" => 5, "Citra" =>
13];
arsort($nilai);
echo array_key_first($nilai);
```

A. Andi
B. Bima
C. Citra
D. 18

8. **Output kode ini:**

```
$input = [];
echo $input["q"] ?? "kosong";
```

A. kosong
B. null
C. Warning: Undefined array key
D. q

Kunci & Pembahasan · PHP Bab 3

← Kembali ke soal · Awal bab

1. **A. 5 15** — $l = 1 adalah nilai default yang dipakai jika argumen kedua tidak dikirim. luas(5) = 5 × 1, luas(5, 3) = 15.
   - **B:** Argumen kedua di pemanggilan kedua menggantikan default.
   - **C:** Pemanggilan pertama memakai default 1.
   - **D:** Parameter default adalah fitur valid.

2. **A. 22 14** — array_sum menjumlah semua elemen; max mengambil yang terbesar. PHP punya puluhan fungsi array_* bawaan yang siap pakai.
   - **B:** max mengambil nilai terbesar.
   - **C:** 3 adalah count, bukan jumlah.
   - **D:** Angka dijumlah, bukan disambung.

3. **A. 3** — array_filter + arrow function fn() (PHP 7.4+) — padanan filter di JavaScript dan list comprehension di Python. Yang lolos: 14, 6, 7.
   - **B:** 6 juga memenuhi ≥ 6.
   - **C:** 1,5 dan 0,75 tidak lolos.
   - **D:** Yang dihitung banyaknya, bukan jumlahnya.

4. **A. Rina, Tono** — array_map menerapkan fungsi ke setiap elemen (seperti map di JavaScript). ucfirst mengkapitalkan huruf pertama. implode menggabungkan array menjadi teks dengan pemisah — kebalikannya explode.
   - **B:** ucfirst mengubah huruf pertama menjadi kapital.
   - **C:** Itu hasil strtoupper.
   - **D:** Pemisahnya koma + spasi.

5. **A. 8 JAKARTA** — strlen = panjang teks (seperti LEN di Excel), strtoupper = huruf besar (seperti UPPER). Nama fungsi PHP banyak yang diawali str_ / str.
   - **B:** strtoupper mengubah semua huruf.
   - **C:** 'Keyboard' terdiri dari 8 huruf.
   - **D:** 'Keyboard' terdiri dari 8 huruf.

6. **A. 1.234.567** — number_format(angka, desimal, pemisah_desimal, pemisah_ribuan). Dengan pemisah ribuan '.', hasilnya format Indonesia. Tambahkan 'Rp ' sendiri di depannya.
   - **B:** Itu jika memakai pemisah default (koma).
   - **C:** number_format menambahkan pemisah ribuan.
   - **D:** Simbol Rp tidak ditambahkan otomatis.

7. **A. Andi** — arsort mengurutkan nilai dari besar ke kecil sambil MEMPERTAHANKAN kuncinya. array_key_first mengambil kunci pertama → sales dengan nilai terbesar. Varian lain: sort/rsort (tanpa kunci), asort (naik), ksort (berdasarkan kunci).
   - **B:** Bima paling kecil.
   - **C:** Citra di urutan kedua.
   - **D:** Yang diambil kuncinya, bukan nilainya.

8. **A. kosong** — Operator ?? (null coalescing) mengembalikan sisi kanan jika sisi kiri tidak ada/null — tanpa warning. Pola wajib saat membaca input form: $_GET['q'] ?? ''.
   - **B:** ?? mengganti nilai yang tidak ada dengan 'kosong'.
   - **C:** ?? mencegah warning tersebut.
   - **D:** q adalah nama kuncinya, bukan nilainya.

PHP untuk Web: Form, Session & Keamanan

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
<form method="post" action="simpan.php">
  <input name="nama">      <!-- name wajib agar terbaca
di $_POST -->
  <button>Simpan</button>
</form>

// simpan.php
session_start();
$nama = trim($_POST["nama"] ?? "");
if ($nama === "") { … tampilkan error … }
// … simpan ke database dengan prepared statement …
header("Location: daftar.php");   // Post/Redirect/Get
exit;
```

Tiga aturan keamanan wajib:

1. **Escape output**: `htmlspecialchars($teks)` → mencegah XSS.
2. **Prepared statement** untuk SQL → mencegah SQL injection.
3. **password_hash / password_verify** → jangan simpan password apa adanya (dan jangan pakai md5).

🧠 Uji Diri — 8 soal

1. **Beda $_GET dan $_POST:**

   A. $_GET membaca data dari URL (?q=laptop), cocok untuk pencarian. $_POST membaca data form yang dikirim di badan request, cocok untuk menyimpan/login
   B. Keduanya sama persis
   C. $_POST lebih lambat
   D. $_GET hanya untuk gambar

2. **Output kode ini, dan kenapa penting?**

```
$komentar = "<script>alert(1)</script>";
echo htmlspecialchars($komentar);
```

   A. <script>alert(1)</script> — tag diubah menjadi teks biasa sehingga tidak dijalankan browser (mencegah XSS)
   B. <script>alert(1)</script> — script dijalankan
   C. Teks kosong
   D. Error

3. **Cara menyimpan password pengguna yang benar di PHP:**

   A. password_hash() saat mendaftar, password_verify() saat login
   B. Simpan apa adanya (plain text)
   C. md5($password)
   D. base64_encode($password)

4. **Kenapa kode ini BERBAHAYA?**

```
$kota = $_GET["kota"];
$sql = "SELECT * FROM pelanggan WHERE kota =
'$kota'";
```

   A. Rentan SQL injection: input seperti ' OR '1'='1 bisa mengubah query. Gunakan prepared statement
   B. Tidak berbahaya
   C. Karena memakai SELECT *
   D. Karena $_GET lambat

5. **Agar pengguna tetap 'login' saat berpindah halaman, PHP memakai...**

   A. Session: session_start() lalu simpan data di $_SESSION
   B. Variabel biasa
   C. Menyimpan password di URL
   D. echo

6. **Field form <input type="text"> tidak muncul di $_POST. Penyebab paling umum:**

   A. Input tidak punya atribut name — PHP membaca data berdasarkan name
   B. Input harus berwarna
   C. Form harus berada di file .php
   D. Browser tidak mendukung POST

7. **Setelah menyimpan data dari form POST, praktik terbaik adalah…**

   A. Redirect dengan header("Location: daftar.php") agar refresh tidak mengirim ulang data (pola Post/Redirect/Get)
   B. Menampilkan halaman yang sama
   C. Menyuruh pengguna menekan back
   D. Menghapus session

8. **Fungsi PHP untuk mengubah < dan > menjadi teks aman sebelum ditampilkan: _____($teks)**

   Jawab: _____

Kunci & Pembahasan · PHP Bab 4

← Kembali ke soal · Awal bab

1. **A. $_GET membaca data dari URL (?q=laptop), cocok untuk pencarian. $_POST membaca data form yang dikirim di badan request, cocok untuk menyimpan/login** — Data GET terlihat di URL dan bisa di-bookmark. Data POST tidak tampil di URL (tapi tetap bukan berarti terenkripsi — pakai HTTPS).
   - **B:** Cara pengirimannya berbeda.
   - **C:** Perbedaan kecepatannya tidak berarti.
   - **D:** GET bisa membawa data apa pun lewat URL.

2. **A. <script>alert(1)</script> — tag diubah menjadi teks biasa sehingga tidak dijalankan browser (mencegah XSS)** — Menampilkan input pengguna tanpa htmlspecialchars membuka celah XSS: orang lain bisa menyisipkan JavaScript ke halamanmu. Aturan: escape setiap data dari pengguna saat menampilkannya. (Fungsi esc() di KodeQuiz melakukan hal yang sama.)
   - **B:** Itu yang terjadi TANPA htmlspecialchars — berbahaya.
   - **C:** Teksnya tetap ditampilkan, hanya aman.
   - **D:** Fungsi ini valid.

3. **A. password_hash() saat mendaftar, password_verify() saat login** — password_hash memakai algoritma lambat yang dirancang khusus untuk password (bcrypt/argon2) dan menambahkan salt otomatis. md5 sudah tidak aman untuk password, dan base64 hanya encoding yang bisa dibalik.
    - **B:** Jika database bocor, semua password langsung terbaca.
    - **C:** md5 terlalu cepat dan rentan ditebak.
    - **D:** base64 bukan enkripsi; bisa langsung didekode.
4. **A. Rentan SQL injection: input seperti ' OR '1'='1 bisa mengubah query. Gunakan prepared statement** — Jangan pernah menyambung input pengguna langsung ke query SQL. Prepared statement mengirim query dan datanya secara terpisah, sehingga input selalu dianggap data, bukan perintah.
    - **B:** Ini salah satu celah keamanan paling berbahaya dan paling umum.
    - **C:** SELECT * kurang efisien, tapi bukan masalah keamanannya.
    - **D:** Kecepatan bukan masalahnya.

5. **A. Session: session_start() lalu simpan data di $_SESSION** — HTTP tidak 'mengingat' apa pun antar request. Session menyimpan data di server dan mengenali pengguna lewat cookie ID sesi. session_start() harus dipanggil sebelum ada output apa pun.
   - **B:** Variabel biasa hilang setelah halaman selesai diproses.
   - **C:** Sangat tidak aman.
   - **D:** echo hanya menampilkan teks.

6. **A. Input tidak punya atribut name — PHP membaca data berdasarkan name** — <input name="nama"> → dibaca dengan $_POST['nama']. Atribut id dipakai CSS/JavaScript, sedangkan name dipakai saat data dikirim ke server.
   - **B:** Warna tidak berpengaruh.
   - **C:** Form HTML biasa pun bisa mengirim ke file PHP.
   - **D:** Semua browser mendukung POST.

7. **A. Redirect dengan header("Location: daftar.php") agar refresh tidak mengirim ulang data (pola Post/Redirect/Get)** — Tanpa redirect, menekan refresh akan menanyakan 'kirim ulang form?' dan data bisa tersimpan dua kali (misal transaksi ganda).
   - **B:** Refresh bisa mengirim ulang data.
   - **C:** Pengalaman pengguna buruk dan tetap berisiko dobel.
   - **D:** Tidak berhubungan.

8. **htmlspecialchars** — htmlspecialchars mencegah XSS. Aturan emas keamanan web: escape output, validasi input, dan pakai prepared statement untuk database.

OOP, Database (PDO) & Laravel

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
class Produk {
    public function __construct(public string $nama,
public int $harga) {}
    public function total(int $qty): int { return
$this->harga * $qty; }
}
$p = new Produk("Mouse", 150000);
echo $p->total(3);            // 450000  (-> untuk
anggota object)

$stmt = $pdo->prepare("SELECT * FROM pelanggan WHERE
kota = ?");
$stmt->execute([$kota]);
$rows = $stmt->fetchAll();
```

Laravel = framework PHP paling populer, memakai pola **MVC**:

```
Model      → Pelanggan::where('kota', 'Bandung')->get()
(Eloquent ORM)
View       → resources/views/pelanggan.blade.php
(Blade)
Controller → menerima request, memanggil model,
mengirim ke view
```

> 🧭 Konsep MVC dan ORM juga ada di Odoo (model Python, view XML, ORM `search`). Yang kamu pelajari di sini tidak sia-sia walaupun PHP bukan prioritas.

🧠 Uji Diri — 8 soal

1. **Output kode ini (PHP 8):**

```
class Produk {
    public function __construct(
        public string $nama,
        public int $harga
    ) {}

    public function total(int $qty): int {
        return $this->harga * $qty;
    }
}

$p = new Produk("Mouse", 150000);
echo $p->total(3);
```

   A. 450000
   B. 150000
   C. Mouse
   D. 3

2. **Di PHP, operator untuk mengakses property/method sebuah object adalah...**

   A. → (contoh: $p→nama)
   B. . (contoh: $p.nama)
   C. :: untuk semua kasus
   D. ⇒

3. **Kenapa kode PDO ini aman dari SQL injection?**

```
$stmt = $pdo->prepare("SELECT * FROM pelanggan
WHERE kota = ?");
$stmt->execute([$kota]);
$hasil = $stmt->fetchAll();
```

   A. Query dan data dikirim terpisah (parameter binding); input selalu dianggap data, bukan bagian dari perintah SQL
   B. Karena memakai fetchAll
   C. Karena tanda tanya menyembunyikan data
   D. Karena PDO mengenkripsi database

4. **Dalam pola MVC (dipakai Laravel), tugas masing-masing bagian:**

   A. Model = data & aturan bisnis, View = tampilan (Blade), Controller = menerima request dan menghubungkan keduanya
   B. Model = tampilan, View = database
   C. Controller = desain CSS
   D. Semua kode ditulis di satu file

5. **Query Eloquent (Laravel) Pelanggan::where('kota', 'Bandung')→get() setara dengan SQL...**

   A. SELECT * FROM pelanggan WHERE kota = 'Bandung'
   B. DELETE FROM pelanggan
   C. INSERT INTO pelanggan
   D. SELECT COUNT(*) FROM pelanggan

6. **Composer di ekosistem PHP berfungsi sebagai...**

   A. Package manager: mengunduh dan mengelola library (seperti pip di Python dan npm di JavaScript)
   B. Editor kode
   C. Server database
   D. Pengganti PHP

7. **Di mana PHP paling banyak dipakai di lowongan kerja Indonesia?**

   A. Web development: Laravel, CodeIgniter, dan WordPress (situs perusahaan, sistem internal, toko online)
   B. Analisis data dan machine learning
   C. Aplikasi iOS
   D. Desain grafis

8. **Variabel khusus untuk merujuk object itu sendiri di dalam method class PHP: $____**

   Jawab: _____

Kunci & Pembahasan · PHP Bab 5

← Kembali ke soal · Awal bab

1. **A. 450000** — Constructor property promotion (PHP 8) membuat dan mengisi property langsung dari parameter constructor. Anggota object diakses dengan →, bukan titik. $this menunjuk object itu sendiri.
   - **B:** Itu harga satuannya.
   - **C:** Method mengembalikan angka.
   - **D:** 3 adalah qty yang dikirim.

2. **A. → (contoh: $p→nama)** — → untuk anggota object, :: untuk anggota static/konstanta class (Produk::PAJAK), dan . untuk menyambung teks. ⇒ dipakai di array asosiatif dan arrow function.
   - **B:** Titik adalah penyambung teks di PHP.
   - **C:** :: khusus untuk anggota static dan konstanta.
   - **D:** ⇒ untuk array asosiatif dan fn().

3. **A. Query dan data dikirim terpisah (parameter binding); input selalu dianggap data, bukan bagian dari perintah SQL** — Tanda ? adalah placeholder. Database menerima struktur query lebih dulu, lalu nilainya. Prinsip ini sama di semua bahasa (Python, Java, Node.js).
   - **B:** fetchAll hanya mengambil hasilnya.
   - **C:** Bukan disembunyikan, tapi dipisahkan dari perintah.
   - **D:** PDO tidak mengenkripsi database.

4. **A. Model = data & aturan bisnis, View = tampilan (Blade), Controller = menerima request dan menghubungkan keduanya** — Memisahkan tanggung jawab membuat aplikasi besar mudah dirawat. Odoo memakai pola yang mirip: model Python, view XML, dan controller untuk route web.
    - **B:** Terbalik.
    - **C:** CSS bagian dari tampilan (View).
    - **D:** Itu yang ingin dihindari MVC.
5. **A. SELECT * FROM pelanggan WHERE kota = 'Bandung'** — Eloquent adalah ORM — sama konsepnya dengan ORM Odoo (search) atau pandas. Pemahaman SQL membuatmu cepat menguasai ORM mana pun, karena ujung-ujungnya ORM menghasilkan SQL.
    - **B:** get() mengambil data, bukan menghapus.
    - **C:** get() tidak menambah data.
    - **D:** Untuk menghitung: →count().
6. **A. Package manager: mengunduh dan mengelola library (seperti pip di Python dan npm di JavaScript)** — composer require nama/paket. Laravel sendiri di-install dengan Composer.
    - **B:** Editor kode misalnya VS Code.
    - **C:** Database misalnya MySQL/PostgreSQL.
    - **D:** Composer berjalan di atas PHP.

7. **A. Web development: Laravel, CodeIgniter, dan WordPress (situs perusahaan, sistem internal, toko online)** — Lowongan PHP/Laravel sangat banyak di Indonesia. Untuk jalur utamamu (Data/BI/Odoo), PHP bukan prioritas — tapi bisa jadi rencana cadangan di jalur web developer.
    - **B:** Data & ML didominasi Python, SQL, dan R.
    - **C:** iOS memakai Swift.
    - **D:** Desain grafis tidak memakai PHP.
8. **this** — $this→harga. Di Python namanya self, di Java/JavaScript this.